

AI 大模型

RAG与智能体开发

当前章节：《前置准备》

通义千问大模型接入



本地部署：Ollama

- Qwen
 - DeepSeek
 - ...
- 1.5b、7b、8b、32b...



阿里云百炼

云端大模型：阿里云百炼平台

- Qwen
 - DeepSeek
 - ...
- 满参数 + 免费额度



接入通义千问大模型：进入阿里云的百炼大模型平台，完成注册，登录，实名（使用支付宝扫一下）。

阿里云百炼

模型应用MCP文档API参考

模型应用

准备工作

获取API Key

配置API Key到环境变量

安装SDK

对话

通义千问

DeepSeek

图像生成

通义万相-文生图V2版

通义万相-文生图V1版

通义万相-通用图像编辑

通义万相-涂鸦作画

通义万相-图像局部重绘

人像风格重绘

图像画面扩展

获取API Key

当您需要通过 API 或 SDK 方式调用大模型及应用时，请先参考本文获取 API Key 作为调用时的鉴权凭证。

开通阿里云百炼的模型服务

使用阿里云主账号前往[阿里云百炼大模型服务平台](#)，如果页面顶部显示以下消息，您需要开通阿里云百炼的模型服务，以获得[新人免费额度](#)。开通阿里云百炼不会产生费用，仅模型调用（超出免费额度后）、模型部署、模型调优模型会产生相应[计费](#)。如果未显示该消息，则表示您已经开通。

新用户开通即享超千万免费tokens

立即开通

如果开通服务时提示“您尚未进行实名认证”，请先进行[实名认证](#)。

获取 API Key

1. 前往 [API-Key](#) 页面，在我的页签（下图位置①）下单击创建我的 **API-KEY**（下图位置②）。

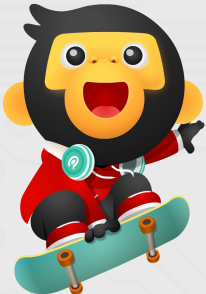
API Key 管理

我的全部

创建我的API-KEY (9/10)

ID	API Key	所归属业务空间	描述	创建时间	操作
2	sk-b****c70d	默认业务空间 (lir	)	2025-05-26 17:35:44	查看删除

2. 在创建新的 **API-KEY** 弹窗中，选择 API Key 归属的业务空间，填写描述信息，并单击确定。



创建API KEY:

阿里云百炼

模型广场

模型体验

文本模型

语音模型

视觉模型

模型应用MCP文档API参考

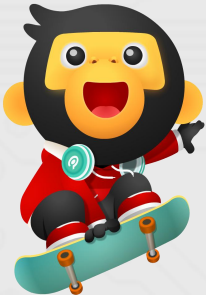
API Key 管理

我的全部

创建我的API-KEY (1/10)

ID	API Key	所归属业务空间	描述	创建时间	操作
2323604	sk-0****378f	默认业务空间 (llm-fiwy9ry8tf7irsux)	演示	2025-07-04 15:00:12	查看 删除

点击查看，就可以复制了

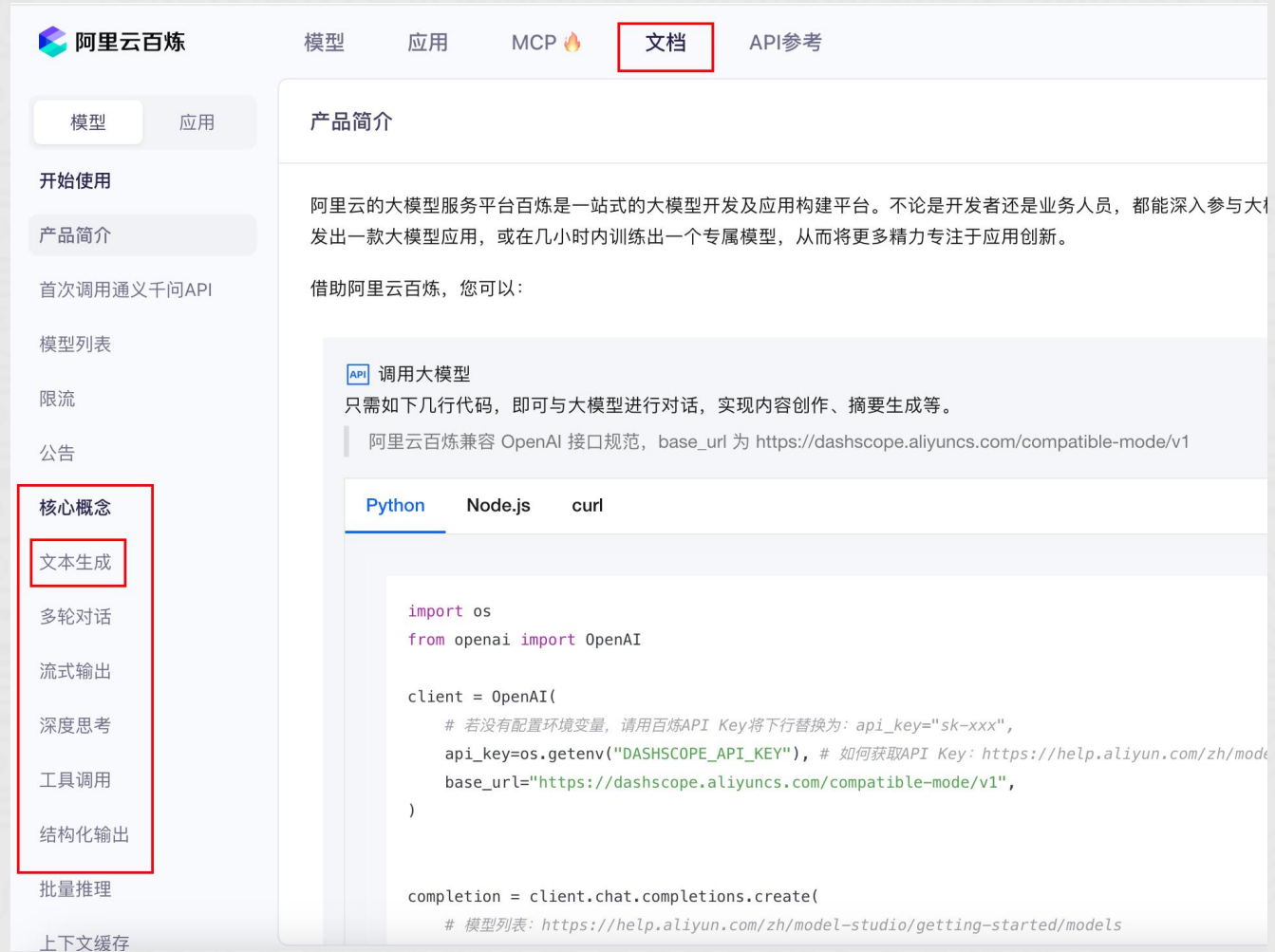


大模型调用准备

安装SDK测试代码：

```
# 基于pip安装openai库  
pip install openai
```

如图所示，获取测试代码



阿里云百炼

模型 应用 MCP 文档 API参考

模型 应用

开始使用

产品简介

首次调用通义千问API

模型列表

限流

公告

核心概念

文本生成

多轮对话

流式输出

深度思考

工具调用

结构化输出

批量推理

上下文缓存

产品简介

阿里云的大模型服务平台百炼是一站式的大模型开发及应用构建平台。不论是开发者还是业务人员，都能深入参与大模型应用开发，发出一款大模型应用，或在几小时内训练出一个专属模型，从而将更多精力专注于应用创新。

借助阿里云百炼，您可以：

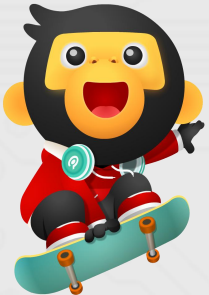
API 调用大模型

只需如下几行代码，即可与大模型进行对话，实现内容创作、摘要生成等。

阿里云百炼兼容 OpenAI 接口规范，base_url 为 <https://dashscope.aliyuncs.com/compatible-mode/v1>

Python Node.js curl

```
import os  
from openai import OpenAI  
  
client = OpenAI(  
    # 若没有配置环境变量，请用百炼API Key将下行替换为: api_key="sk-xxx",  
    api_key=os.getenv("DASHSCOPE_API_KEY"), # 如何获取API Key: https://help.aliyun.com/zh/model-studio/getting-started/models  
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",  
)  
  
completion = client.chat.completions.create(  
    # 模型列表: https://help.aliyun.com/zh/model-studio/getting-started/models
```



大模型调用准备

验证：复制并指向官方提供的调试代码

```
from openai import OpenAI

client = OpenAI(
    api_key="your api key"
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
)

completion = client.chat.completions.create(
    # 模型列表: https://help.aliyun.com/zh/model-studio/getting-started/models
    model="qwen-plus", # qwen-plus 属于 qwen3 模型, 如需开启思考模式, 请参见: https://help.aliyun.com/zh/model-studio/getting-started/models
    messages=[
        {'role': 'system', 'content': 'You are a helpful assistant.'},
        {'role': 'user', 'content': '你是谁? '}
    ]
)
print(completion.choices[0].message.content)
```



大模型调用准备

安全优化：为避免在代码中明文出现APIKEY，可以使用dotenv库改造代码隐藏APIKEY

```
from openai import OpenAI
from dotenv import load_dotenv
import os
# 加载 .env 文件
load_dotenv()

client = OpenAI(
    api_key=os.getenv("api_key")
    base_url=os.getenv("base_url"),
)

completion = client.chat.completions.create(
    # 模型列表: https://help.aliyun.com/zh/model-studio/getting-started/models
    model="qwen-plus", # qwen-plus 属于 qwen3 模型, 如需开启思考模式, 请参见: https://help.aliyun.com/zh/model-studio/getting-started/models
    messages=[
        {'role': 'system', 'content': 'You are a helpful assistant.'},
        {'role': 'user', 'content': '你是谁? '}
    ]
)
print(completion.choices[0].message.content)
```



技术准备

前文代码的messages是一个list，内嵌了多个字典。可以发现字典的结构是：{'role': ... , 'content': ...}
其中role表示的是角色，content表示的是内容。

在chat 大语言模型中，user、assistant、system 是三种核心角色，它们的定位和功能不同，共同构成对话的上下文结构。

System（系统角色）

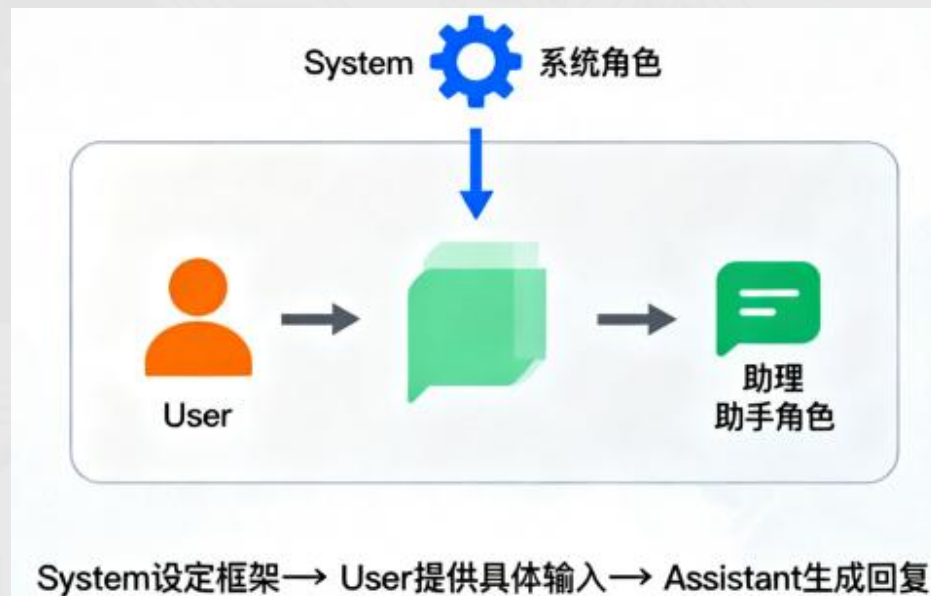
- 定位：控制对话的全局方向和模型的行为模式
- 作用：**设定上下文**：定义对话的背景、目标或规则（例如“你是一个专业翻译，只回答与翻译相关的问题”）。**调整风格**：指定回复的语气（严肃、幽默）、格式（分点、Markdown）或内容限制（避免敏感话题）。**长期控制**：在对话中持续影响模型的输出（即使后续对话中没有重复指令）。
- 典型场景：初始化对话时设定角色（如医生、律师、代码助手）。限制模型的回答范围（如“仅用英文回答”）。设定复杂任务的流程（如分步骤完成任务）。

user

- 定位：代表**真实用户输入**，是驱动对话的核心。
- 作用：提出问题或请求（如“帮我写一首关于夏天的诗”）。提供补充信息（如“上一句翻译成法语”）。修正模型行为（如“用更简单的语言解释”）。
- 典型场景：直接交互：用户提问、追问或反馈。间接控制：通过用户消息调整模型输出（如在消息中附加指令）。

assistant（助手角色）

- 定位：**模型生成的回复内容**。
- 作用：回答问题：基于上下文生成符合用户需求的回复。自我修正：在后续对话中根据用户反馈调整回答（如“抱歉，之前的回答有误，正确的是...”）。遵循指令：执行 system 或 user 指定的规则（如分点回答、使用特定格式）。
- 典型场景：直接生成文本、代码、建议等。通过历史 assistant 消息实现多轮对话连贯性。



AI 大模型

RAG与智能体开发

当前章节：《前置准备》

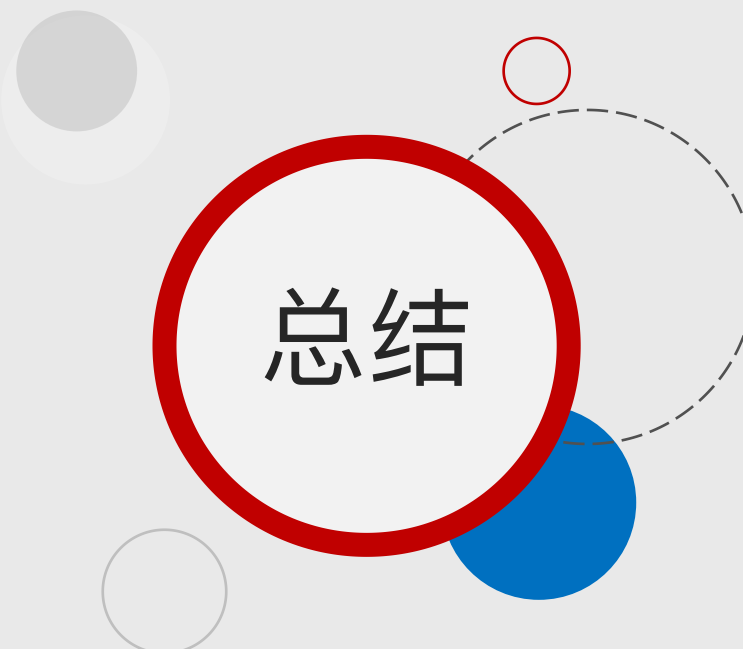
代码调用云端大模型

代码调用云端大模型

通过代码调用阿里云百炼平台上的模型很简单，仅需要：

- 阿里云百炼平台上，创建API-KEY
- 通过pip为Python程序提供 OpenAI 库: `pip install openai`
- 编写代码测试（可从百炼平台官网复制代码，直接执行）





总结

通过代码调用阿里云百炼平台大模型，主要动作

- 准备好API-KEY
- 安装openai库
- 复制官网示例代码并执行（可在代码中替换需要调用的模型）

AI 大模型

RAG与智能体开发

当前章节：《前置准备》

使用环境变量保护 APIKEY

代码调用云端大模型

如下代码，将APIKEY明文显示在代码中，是有很大的安全隐患的。我们可以通过环境变量来隐藏明文APIKEY。

```
import os
from openai import OpenAI

client = OpenAI(
    # 若没有配置环境变量 请用百炼API Key将下行替换为: api_key="sk-xxx",
    api_key="sk-c482ad493b4347a9863bad0014354abe",
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
)
completion = client.chat.completions.create(
    model="deepseek-v3",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "你是谁，能做什么? "},
    ],
    stream=True
)
for chunk in completion:
    print(chunk.choices[0].delta.content, end="", flush=True)
```

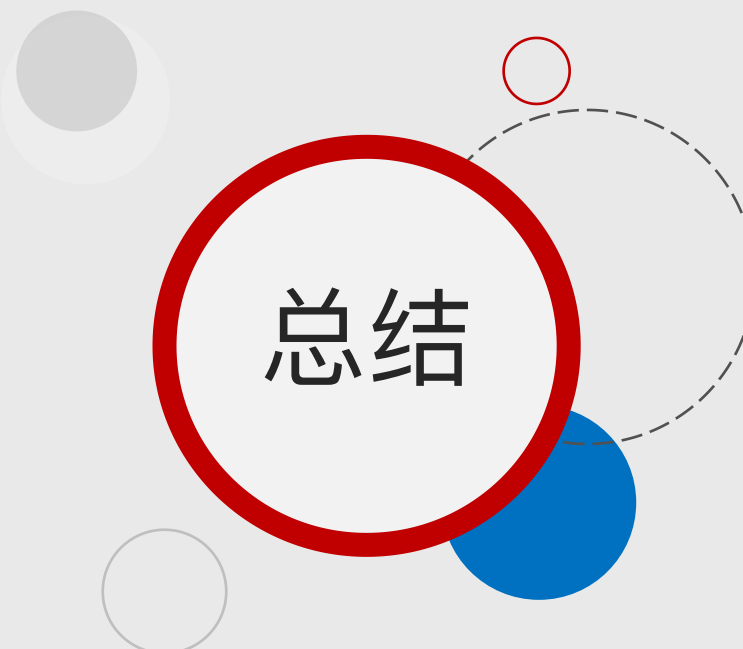
OPENAI_API_KEY

用于openai库，记录APIKEY

DASHSCOPE_API_KEY

用于langchain库（后续学习）





总结

- 将APIKEY暴露在代码中是不合理的
- 可以通过环境变量：OPENAI_API_KEY和DASHSCOPE_API_KEY记录值，代码会自动读取变量从而获取值
- Windows系统通过图形化界面配置环境变量
- Mac系统在终端内修改 .zshrc 文件添加环境变量
 - `export OPENAI_API_KEY=xxx`
 - `export DASHSCOPE_API_KEY=xxx`
- 配置完成后，重启PyCharm生效（如不生效可以重启电脑）

AI 大模型

RAG与智能体开发

当前章节：《前置准备》

Ollama简介

Ollama简介

为了避免未来阿里云免费额度到期或不提供免费活动导致的无法开发，课程额外补充：

基于Ollama部署本地模型，供代码调用。

PS：此为备用方案，课程主体还是基于阿里云百炼平台做开发。

ollama：是一款旨在简化大型语言模型本地部署和运行过程的开源软件。

- ollama提供了一个轻量级、易于扩展的框架，让开发者能够在本地机器上轻松构建和管理LLMs（大型语言模型）
- 通过ollama，开发者可以导入和定制自己的模型，无需关注复杂的底层实现细节。
- 网址：<https://ollama.com>

简单来说可以认为是阿里云百炼平台的本地版，在自己电脑上部署和运行大模型，由自己电脑的硬件提供算力支撑模型运行。



Cloud models are now available in Ollama

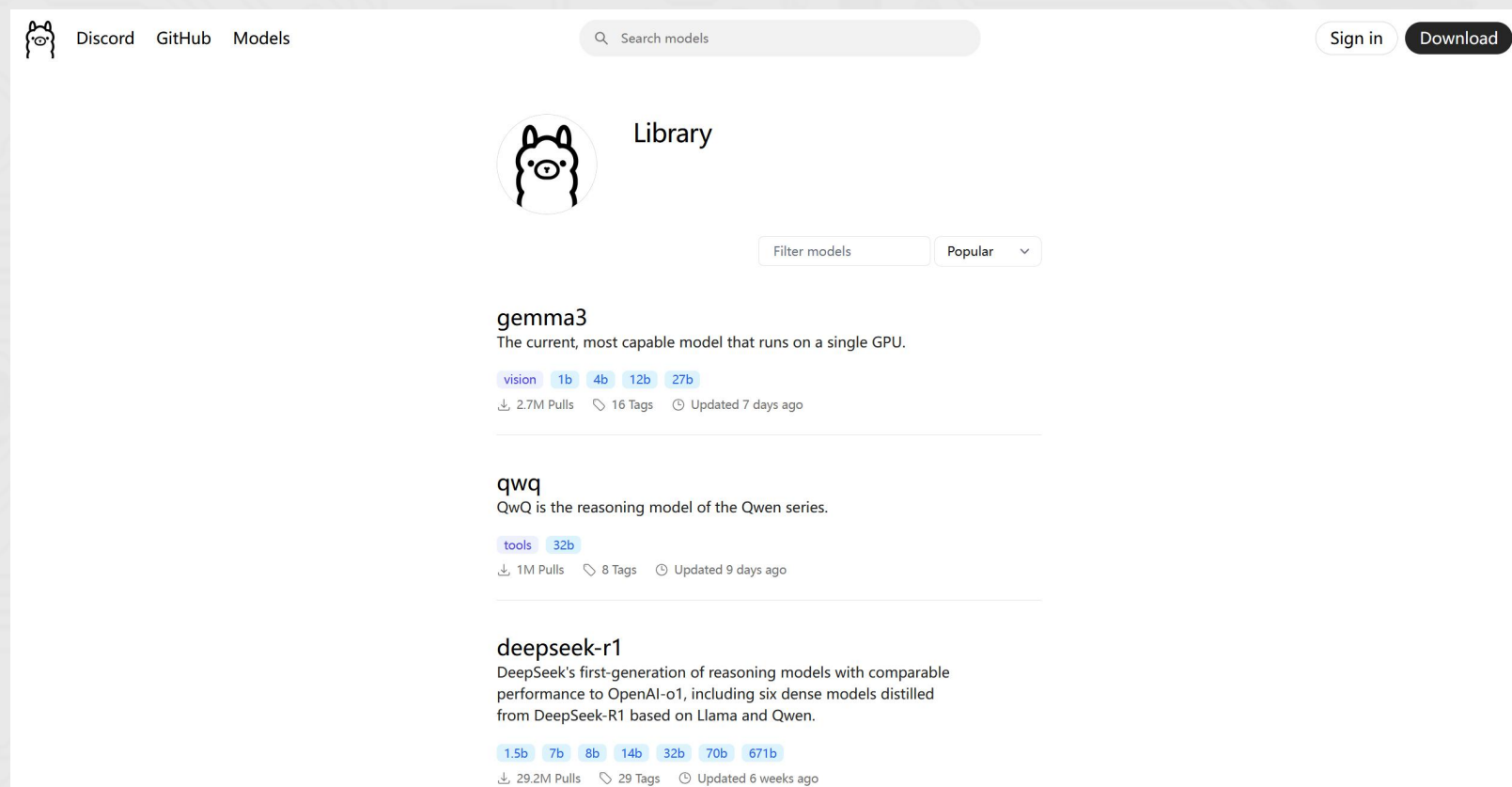
Chat & build with open models

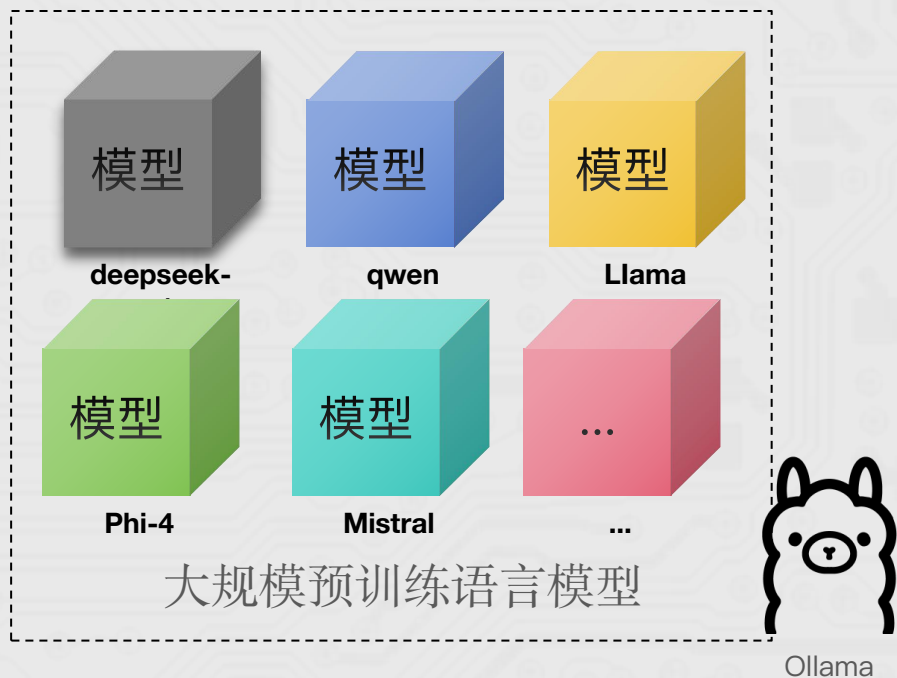
Download

Available for macOS,
Windows, and Linux

ollama 支持多种开源模型，涵盖文本生成、代码生成、多模态推理等场景。用户可以根据需求选择合适的模型，并通过简单的命令行操作在本地运行。

ollama 官方模型库: <https://ollama.com/library>





```
C:\Users\javac>ollama run deepseek-r1:1.5b
```

```
>>> 你是谁
```

```
<think>
```

```
</think>
```

您好！我是由中国的深度求索（DeepSeek）公司开发的智能助手DeepSeek-R1。如您有任何任何问题，我会尽我所能为您提供帮助。

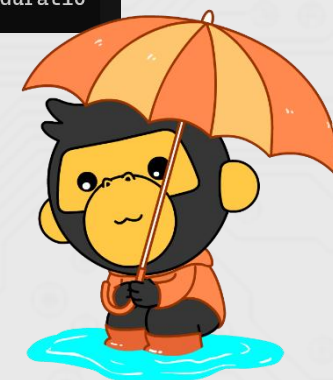
```
>>> Send a message (/? for help)
```

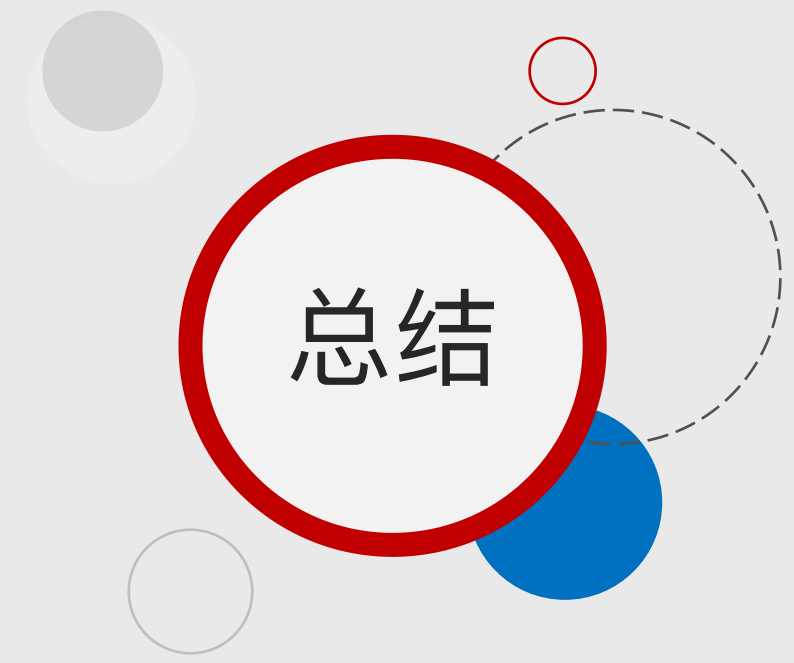
命令行访问

```
C:\Users\javac>curl -X POST -d '{"model":"deepseek-r1:1.5b","messages":[{"role":"user","content":"你好"}],"stream":false}' http://127.0.0.1:11434/api/chat
```

```
{
  "model": "deepseek-r1:1.5b",
  "created_at": "2025-03-27T07:04:18.5956853Z",
  "message": {
    "role": "assistant",
    "content": "\u003cthink\u003e\n\n\u003c/think\u003e\n\n你好！很高兴见到你，有什么我可以帮忙的吗？无论是学习、工作还是生活中的问题，都可以告诉我哦！😊"
  },
  "done_reason": "stop",
  "done": true,
  "total_duration": 615821600,
  "load_duration": 26375300,
  "prompt_eval_count": 4,
  "prompt_eval_duration": 1823400,
  "eval_count": 31,
  "eval_duration": 587106800
}
```

Restful API接口





总结

1.ollama是什么?

是一款旨在简化大型语言模型本地部署和运行过程的开源软件。

2.如何查看ollama支持的模型?

通过ollama 官方模型库: <https://ollama.com/library>

- deepseek-r1,qwen等模型均有提供

AI 大模型

RAG与智能体开发

当前章节：《前置准备》

Windows/Mac系统部署
Ollama

Ollama的部署

Ollama的部署还是很简单的，只需要进入官方网站点击Download按钮下载安装。

安装完成后，通过命令：

- `ollama run 模型名称`

即可运行对应的模型，并在命令行内做交互。

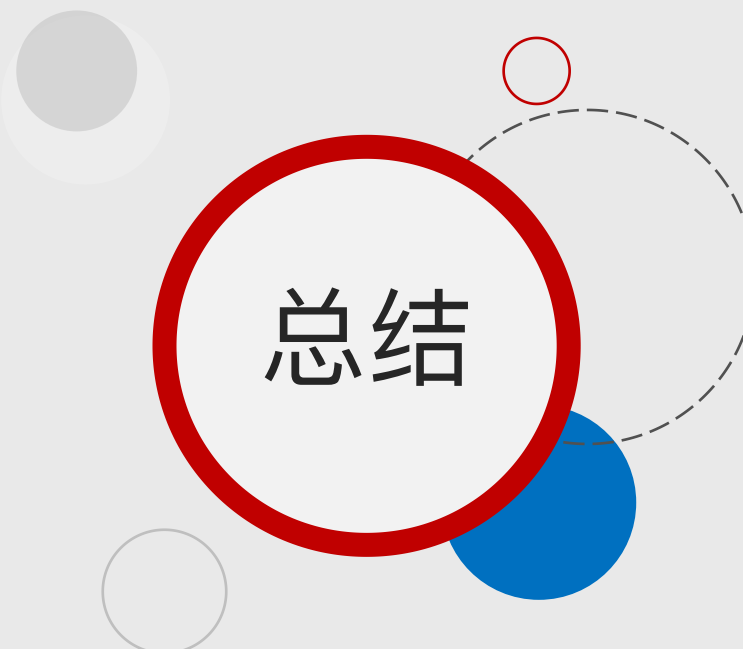


Cloud models are now available in Ollama

Chat & build with open models

Download

Available for macOS,
Windows, and Linux



总结

1. Ollama部署：在官网下载并安装客户端即可。
2. 蒸馏模型就是对标准大模型核心技能的学习，并进行瘦身，从而获得更低性能要求。简单来说蒸馏模型就是标准大模型的学生，学到了老师的核心本领，但没有老师强。根据参数量的不同，参数量越大，蒸馏模型学到老师核心本领就越扎实，性能越好。
3. 参数量选择：
 - 集显：1.5b左右
 - 4G独显：8b以内
 - 8G独显：14b以内
 - ...具体可以自己试一试
 - 参数量越大对硬件要求越高，如果硬件能力不足，模型运算和吐字速度会下降。

AI 大模型

RAG与智能体开发

当前章节：《前置准备》

Ollama的基础命令

Ollama的部署

完成Ollama的部署和使用后，后续如果要管理电脑中的模型，可以使用Ollama命令。

主要介绍如下几个命令：

- ollama list：列出当前已下载的模型
- ollama pull 模型名称：联网下载指定的模型
- ollama run 模型名称：运行指定的模型（如不存在会先下载）
- ollama --help：查看其它可用的命令帮助

PS：上述命令操作，请先运行Ollama

PS：上述命令操作，Windows系统和Mac系统通用



Cloud models are now available in Ollama

Chat & build with open models

Download

Available for macOS,
Windows, and Linux

AI 大模型

RAG与智能体开发

当前章节：《前置准备》

代码调用Ollama的本地模型

代码调用Ollama的本地模型

使用代码调用Ollama的本地模型还是很简单的，只需要将原有代码进行简单改动即可。

```
from openai import OpenAI

client = OpenAI(
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
)
completion = client.chat.completions.create(
    model="deepseek-v3",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "你是谁，能做什么？"},
    ],
    stream=True
)
for chunk in completion:
    print(chunk.choices[0].delta.content, end="", flush=True)
```

- 将base_url改为: <http://localhost:11434/v1>
- 将model改为对应本地模型名称，如: qwen3:4b



AI 大模型

RAG与智能体开发

当前章节：《Python OpenAI库入门》

OpenAI库的基础使用

OpenAI库的基础使用

OpenAI库是OpenAI官方推出的Python SDK，核心作用是让开发者能简单、高效地调用 OpenAI 的各类 API（如 GPT 聊天、DALL·E 绘图、语音转文字等），无需手动处理 HTTP 请求、身份验证等底层细节。

由于其发布较早且比较易用，现如今许多模型服务商（如阿里云百炼平台）均兼容OpenAI SDK的调用。



OpenAI库的基础使用

获取客户端对象

```
from openai import OpenAI

client: OpenAI = OpenAI(
    api_key="your_api_key_here",
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
)
```

主要是用如上2个参数：

- api_key: 模型服务商提供的APIKEY密钥
- base_url: 模型服务商的API接入地址
 - 主要基于此参数来切换不同的模型服务商（如OpenAI、阿里云、腾讯云等）



OpenAI库的基础使用

调用模型

```
from openai.types.chat.chat_completion import ChatCompletion

response: ChatCompletion = client.chat.completions.create(

    model="qwen3-max",

    messages=[

        {"role": "system", "content": "你是一个Python编程专家。"},

        {"role": "assistant", "content": "我是一个Python编程专家。请问有什么可以帮助您的吗？"},

        {"role": "user", "content": "for循环输出1到5的数字"}

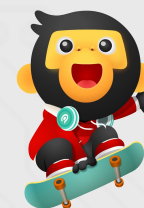
    ]

)
```

client.chat.completions.create创建ChatCompletion对象

主要参数有2个：

- model：选择所用模型，如代码的qwen3-max
- messages：提供给模型的消息
 - 类型：list，可以包含多个字典消息
 - 每个字典消息包含2个key
 - role：角色
 - content：内容
- system角色：设定助手的整体行为、角色和规则，为对话提供上下文框架（如指定助手身份、回答风格、核心要求），是全局的背景设定，影响后续所有交互。
- assistant角色：代表 AI 助手的回答，可以在代码中认为设定
- user角色：代表用户，发送问题、指令或需求



OpenAI库的基础使用

处理结果

response变量就是ChatCompletion对象，其包含信息如下所示

```
{
  "id": "chatcmpl-xxxx",
  "object": "chat.completion",
  "created": 1735689600,
  "model": "gpt-3.5-turbo-0125",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "生成的回复内容"
      },
      "finish_reason": "stop" # stop=正常结束, length=令牌数超限, function_call=触发函数调用
    }
  ],
  "usage": { # 令牌消耗统计
    "prompt_tokens": 50,
    "completion_tokens": 80,
    "total_tokens": 130
  }
}
```

可以通过

```
print(response.choices[0].message.content)
```

输出模型给出的回答信息



OpenAI库的基础使用

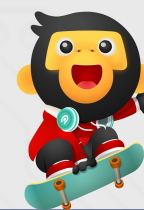
完整的基础代码

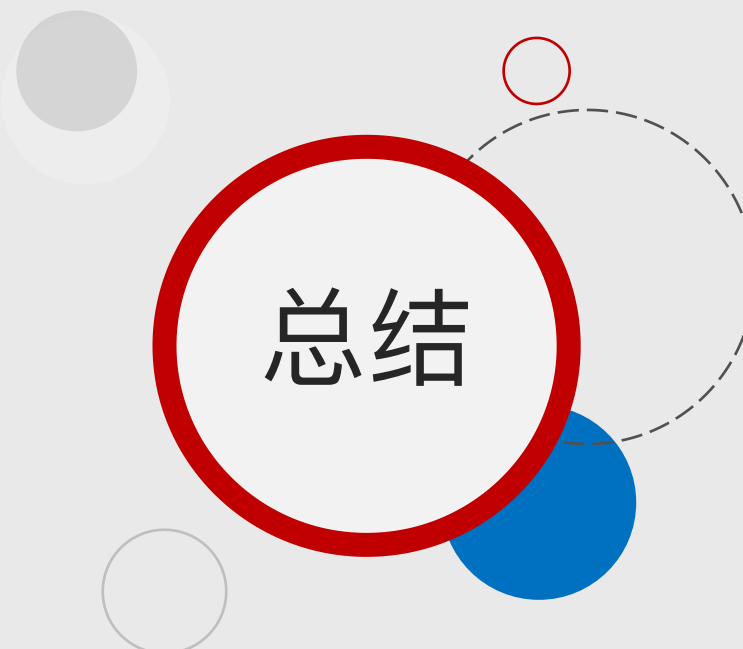
```
from openai import OpenAI
from openai.types.chat.chat_completion import ChatCompletion

client: OpenAI = OpenAI(
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
)

response: ChatCompletion = client.chat.completions.create(
    model="qwen3-max",
    messages=[
        {"role": "system", "content": "你是一个Python编程专家。"},
        {"role": "assistant", "content": "我是一个Python编程专家。请问有什么可以帮助您的吗?"},
        {"role": "user", "content": "for循环输出1到5的数字"}
    ]
)

print(response.choices[0].message.content)
```





总结

OpenAI库是OpenAI公司发布的Python SDK，方便与编程调用其产品，现许多模型服务商都兼容OpenAI SDK的调用。

使用主要就3个流程：

- 创建客户端对象（OpenAI类对象）
- 和模型对话（`client.chat.completions.create`），可以提供3个角色使用
 - system：设定模型的行为和规则
 - assistant：设定模型的回答，由用户设定
 - user：用户的提问
- 处理结果：`response.choices[0].message.content`

AI 大模型

RAG与智能体开发

当前章节：《Python OpenAI库入门》

OpenAI库的流式输出

OpenAI库的流式输出

可以设定结果输出为stream模式（流式输出），获得更好的使用体验。

开启流式输出主要就2步：

1. 在client.chat.completions.create()调用模型的时候设定参数：stream=True
2. for循环response对象，并在循环内输出内容

```
from openai import OpenAI
from openai.types.chat.chat_completion import ChatCompletion

client: OpenAI = OpenAI(
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
)

response: ChatCompletion = client.chat.completions.create(
    model="qwen3-max",
    messages=[
        {"role": "system", "content": "你是一个Python编程专家。"},
        {"role": "assistant", "content": "我是一个Python编程专家。请问有什么可以帮助您的吗？"},
        {"role": "user", "content": "for循环输出1到5的数字"}
    ],
    stream=True      # 开启流式输出功能
)

for chunk in response:
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end='', flush=True)
```



AI 大模型

RAG与智能体开发

当前章节：《Python OpenAI库入门》

OpenAI库
附带历史消息调用模型

OpenAI库附带历史消息调用模型

调用模型传入的参数messages，其要求是list对象，即表明其支持非常多的消息在内。

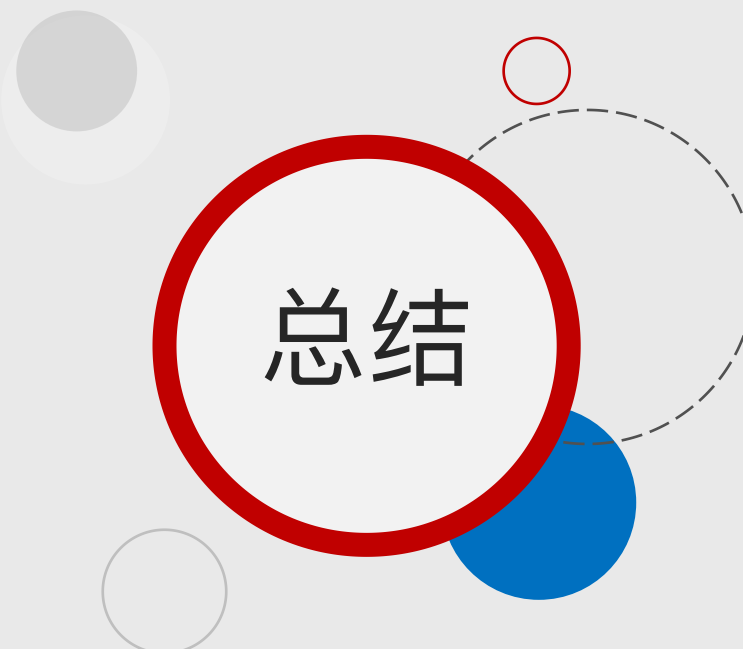
我们可以基于此，将历史消息填入，让模型知晓对话的上下文，更好的回答

```
from openai import OpenAI

# 1. 获取client对象，OpenAI类对象
client = OpenAI(
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
)

# 2. 调用模型
response = client.chat.completions.create(
    model="qwen3-max",
    messages=[
        {"role": "system", "content": "你是AI助理，回答很简洁"},
        {"role": "user", "content": "小明有2条宠物狗"},
        {"role": "assistant", "content": "好的"},
        {"role": "user", "content": "小红有3只宠物猫"},
        {"role": "assistant", "content": "好的"},
        {"role": "user", "content": "总共有几个宠物? "}
    ],
    stream=True  # 开启了流式输出的功能
)

# 3. 处理结果
# print(response.choices[0].message.content)
for chunk in response:
    print(
        chunk.choices[0].delta.content,
        end=" ",  # 每一段之间以空格分隔
        flush=True  # 立刻刷新缓冲区
    )
```



总结

我们可以在调用模型的时候，通过：

- 在messages的list内，组织历史消息提供给模型

PS：当前的历史消息是一次性的，如果是生产系统可以将消息保存到文件、数据库等持久化工具内，需要的时候提取使用

后续学习LangChain库，会学习短期记忆和长期记忆的使用方法。

AI 大模型

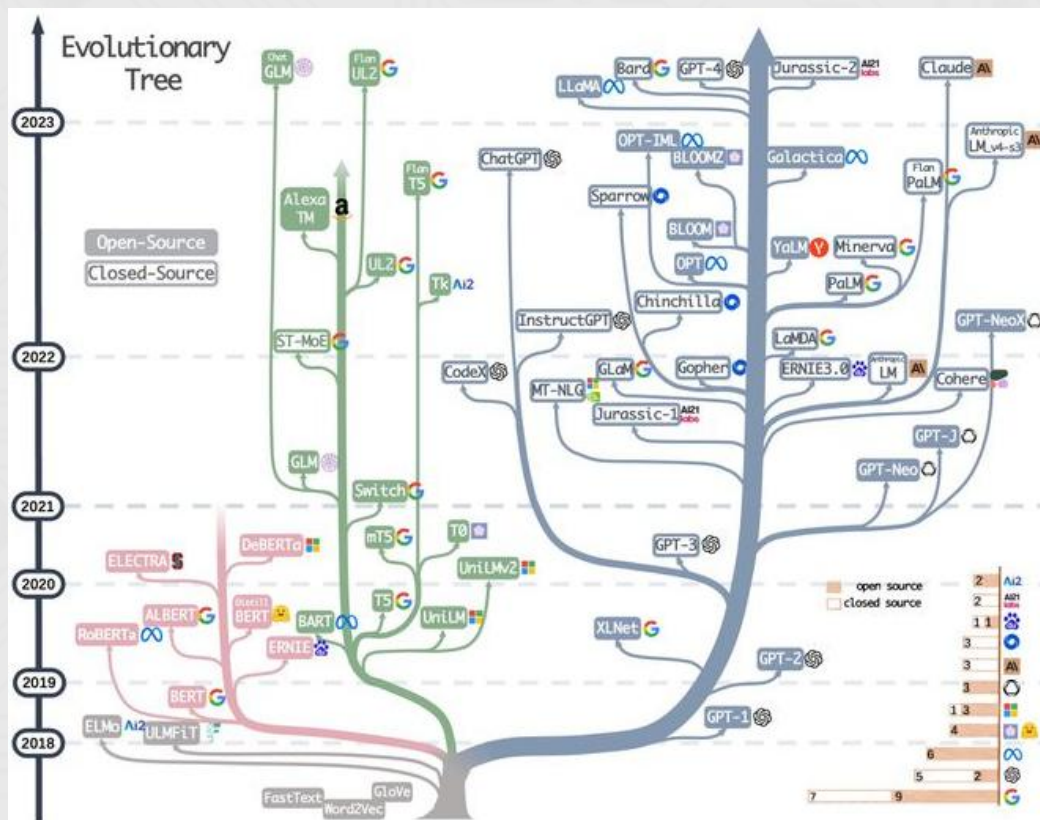
RAG与智能体开发

当前章节：《提示词工程》

LLM介绍

LLM介绍

大语言模型 (英文: Large Language Model, 缩写LLM) 是一种人工智能模型, 旨在理解和生成人类语言. 大语言模型可以处理多种自然语言任务, 如文本分类、问答、翻译、对话等等.



截止目前, 语言模型发展走过了三个阶段:

- **第一阶段:** 设计一系列的自监督训练目标 (MLM、NSP等), 设计新颖的模型架构 (Transformer), 遵循Pre-training和Fine-tuning范式。典型代表是BERT、GPT、XLNet等;
- **第二阶段:** 逐步扩大模型参数和训练语料规模, 探索不同类型的架构。典型代表是BART、T5、GPT-3等;
- **第三阶段:** 走向AIGC (Artificial Intelligent Generated Content) 时代, 模型参数规模步入千万亿, 模型架构为自回归架构, 大模型走向对话式、生成式、多模态时代, 更加注重与人类交互进行对齐, 实现可靠、安全、无毒的模型。典型代表是InstructionGPT、ChatGPT、Bard、GPT-4等。

语言模型 (Language Model, LM)

语言模型 (Language Model) 旨在建模词汇序列的生成概率，提升机器的语言智能水平，使机器能够模拟人类说话、写作的模式进行自动文本输出。

通俗理解: 用来计算一个句子的概率的模型，也就是判断一句话是否是人话的概率。

标准定义: 对于某个句子序列, 如 $S = \{W_1, W_2, W_3, \dots, W_n\}$, 语言模型就是计算该序列发生的概率, 即 $P(S)$ 。如果给定的词序列符合语用习惯, 则给出高概率, 否则给出低概率。



语言模型技术的发展四阶段

基于规则和统计的语言模型

神经语言模型

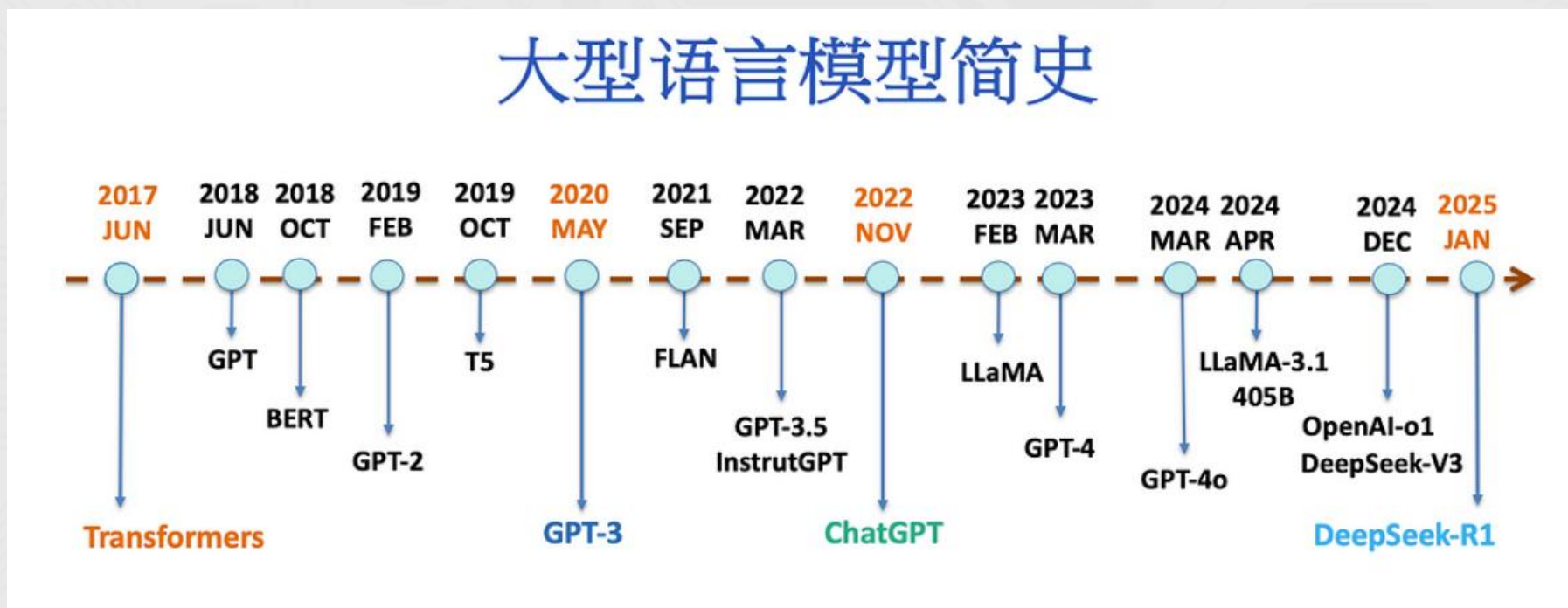
预训练语言模型

大语言模型

LLM主要类别架构

LLM本身基于transformer架构。自2017年，attention is all you need诞生起，原始的transformer模型为不同领域的模型提供了灵感和启发。

基于原始的Transformer框架，衍生出了一系列模型，一些模型仅仅使用encoder或decoder，有些模型同时使用encoder+decoder。



LLM分类一般分为三种：自编码模型（encoder）、自回归模型(decoder)和序列到序列模型(encoder-decoder)。

自编码模型 (AutoEncoder model, AE)

AE模型，代表作BERT，其特点为：Encoder-Only，基本原理：是在输入中随机MASK掉一部分单词，根据上下文预测这个词。AE模型通常用于内容理解任务，比如自然语言理解（NLU）中的分类任务：情感分析、提取式问答。

代表模型 BERT

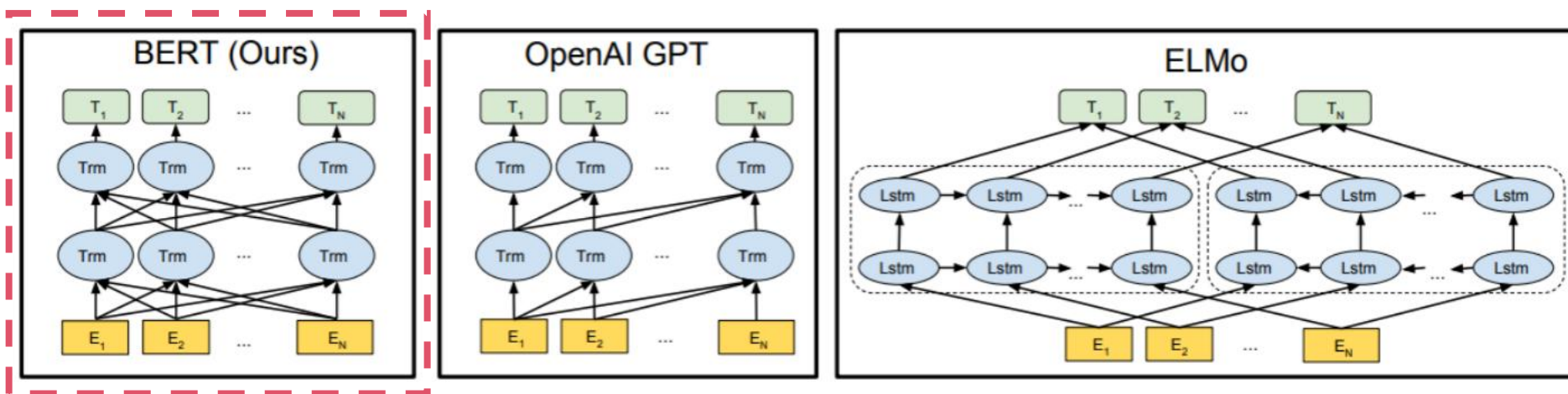


Figure 3: Differences in pre-training model architectures. BERT uses a bidirectional Transformer. OpenAI GPT uses a left-to-right Transformer. ELMo uses the concatenation of independently trained left-to-right and right-to-left LSTMs to generate features for downstream tasks. Among the three, only BERT representations are jointly conditioned on both left and right context in all layers. In addition to the architecture differences, BERT and OpenAI GPT are fine-tuning approaches, while ELMo is a feature-based approach.

自回归模型 (Autoregressive model, AR)

AR模型，代表作GPT，其特点为：Decoder-Only，基本原理：从左往右学习的模型，只能利用上文或者下文的信息，比如：AR模型从一系列time steps中学习，并将上一步的结果作为回归模型的输入，以预测下一个time step的值。

AR模型通常用于生成式任务，在长文本的生成能力很强，比如自然语言生成（NLG）领域的任务：摘要、翻译或抽象问答。

代表模型 GPT

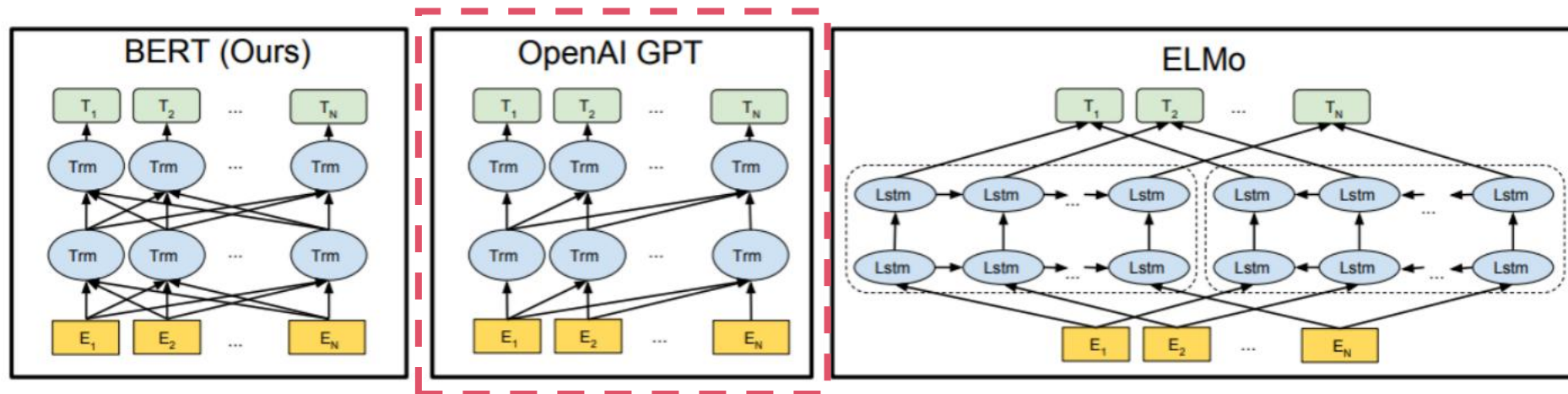


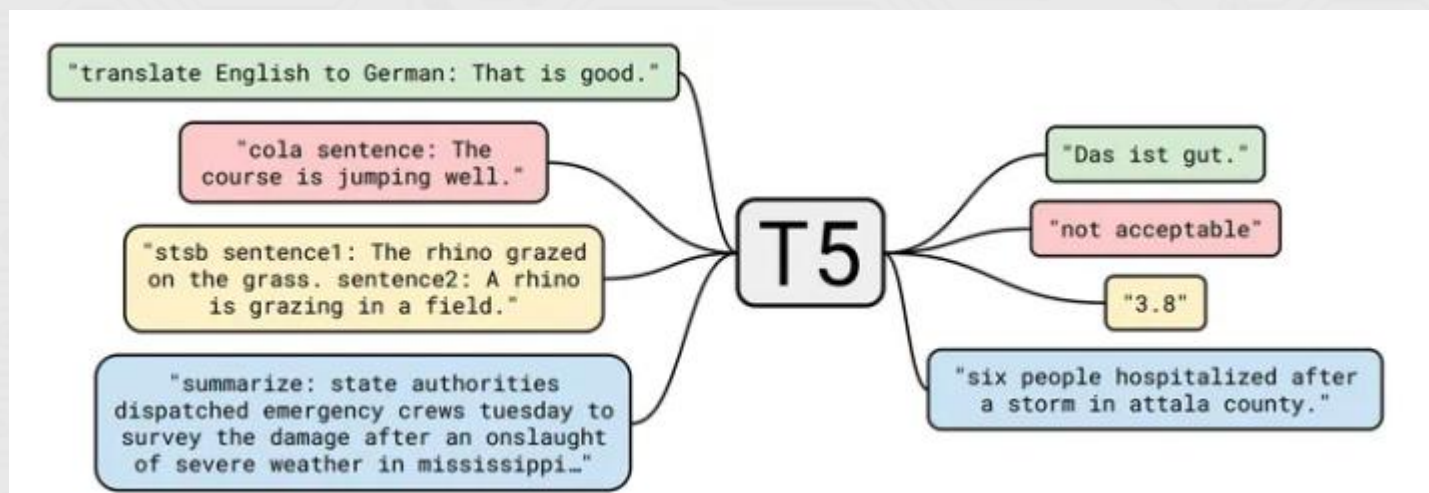
Figure 3: Differences in pre-training model architectures. BERT uses a bidirectional Transformer. OpenAI GPT uses a left-to-right Transformer. ELMo uses the concatenation of independently trained left-to-right and right-to-left LSTMs to generate features for downstream tasks. Among the three, only BERT representations are jointly conditioned on both left and right context in all layers. In addition to the architecture differences, BERT and OpenAI GPT are fine-tuning approaches, while ELMo is a feature-based approach.

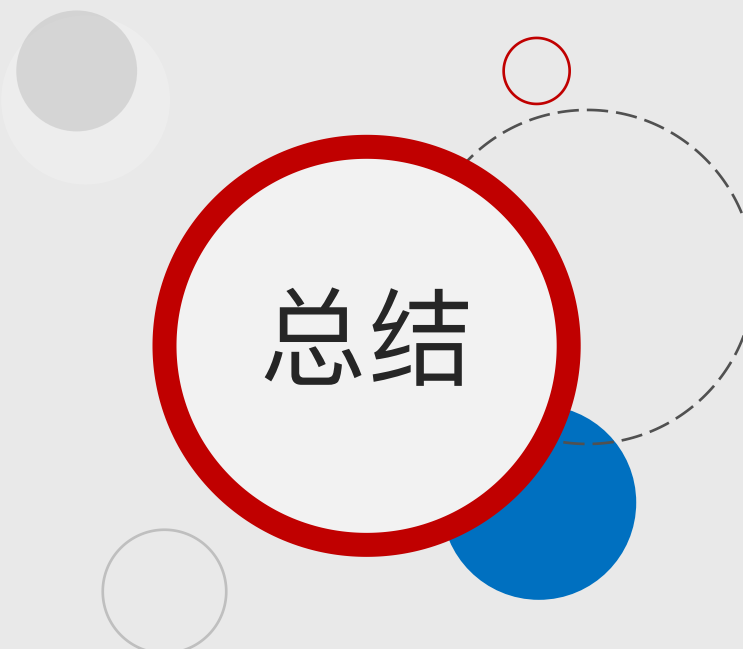
序列到序列 (Sequence to Sequence Model)

encoder-decoder模型同时使用编码器和解码器。它将每个task视作序列到序列的转换/生成（比如，文本到文本，文本到图像或者图像到文本的多模态任务）。对于文本分类任务来说，编码器将文本作为输入，解码器生成文本标签。

Encoder-decoder模型通常用于需要内容理解和生成的任务，比如机器翻译。

代表模型T5





总结

大语言模型 (英文: Large Language Model, 缩写LLM) 是一种人工智能模型, 旨在理解和生成人类语言.

语言模型技术的发展四阶段:

- 基于规则和统计的语言模型
- 神经语言模型
- 预训练语言模型
- 大语言模型

大语言模型主要分为3类: 自编码模型 (encoder, 代表: BERT)、自回归模型(decoder, 代表: GPT)和序列到序列模型(encoder-decoder, 代表: T5)。

AI 大模型

RAG与智能体开发

当前章节：《提示词工程》

主流LLM介绍

LLM主流大模型类别

随着ChatGPT迅速火爆，引发了大模型的时代变革，国内外各大公司也快速跟进生成式AI市场，近百款大模型发布及应用。目前，比较流行的开源的大语言模型主要以下几种：LLaMA大模型、ChatGLM大模型、Qwen大模型、DeepSeek大模型

模型大小	训练显存需求	推理显存需求	CPU内存需求	计算资源	训练时间	多GPU需求	推荐使用场景
7B	20–24GB (单卡)	10–14GB (FP16)	16–32GB	单卡 (A100/A6000)	数天至数周	可选 (单卡/双卡)	中小企业应用部署
13B	40–48GB (双卡)	20–26GB (FP16)	32–64GB	双卡 (A100 40G)	1–2周	双卡以上	企业级应用、中等负载任务
32B	160–200GB (多卡)	64–70GB (FP16)	64–128GB	4–8卡 (A100/H100)	2–4周	必需 (4+卡)	复杂NLP任务、云端服务
70B	400GB+ (多卡集群)	140GB+ (FP16)	128–256GB	8–16卡 (H100 集群)	1–3个月	必需 (16+卡)	超大规模任务、行业解决方案
304B更大	1.5TB+ (分布式)	600GB+ (需量化)	512GB+	数百卡 (超算集群)	数月以上	必需 (百卡级)	国家/科研级超级计算

LLaMA模型

LLaMA (Large Language Model Meta AI) , 由 Meta AI 于2023年发布的一个开放且高效的大型基础语言模型, 共有 7B、13B、33B、65B (650 亿) 四种版本。

LLaMA训练数据是以英语为主的拉丁语系, 另外还包含了来自 GitHub 的代码数据。

- 训练数据以英文为主, 不包含中韩日文, 所有训练数据都是开源的。
- 其中LLaMA-65B 和 LLaMA-33B 是在 1.4万亿 (1.4T) 个 token 上训练的
- 而最小的模型 LLaMA-7B 和LLaMA-13B 是在 1万亿 (1T) 个 token 上训练的。



ChatGLM模型

ChatGLM 是清华大学提出的一个开源、支持中英双语的对话语言模型。

该模型使用了和 ChatGPT 相似的技术

- 经过约 1T 标识符的中英双语训练(中英文比例为 1:1)
- 辅以监督微调、反馈自助、人类反馈强化学习等技术的加持
- 62 亿参数的 ChatGLM-6B 已经能生成相当符合人类偏好的回答。

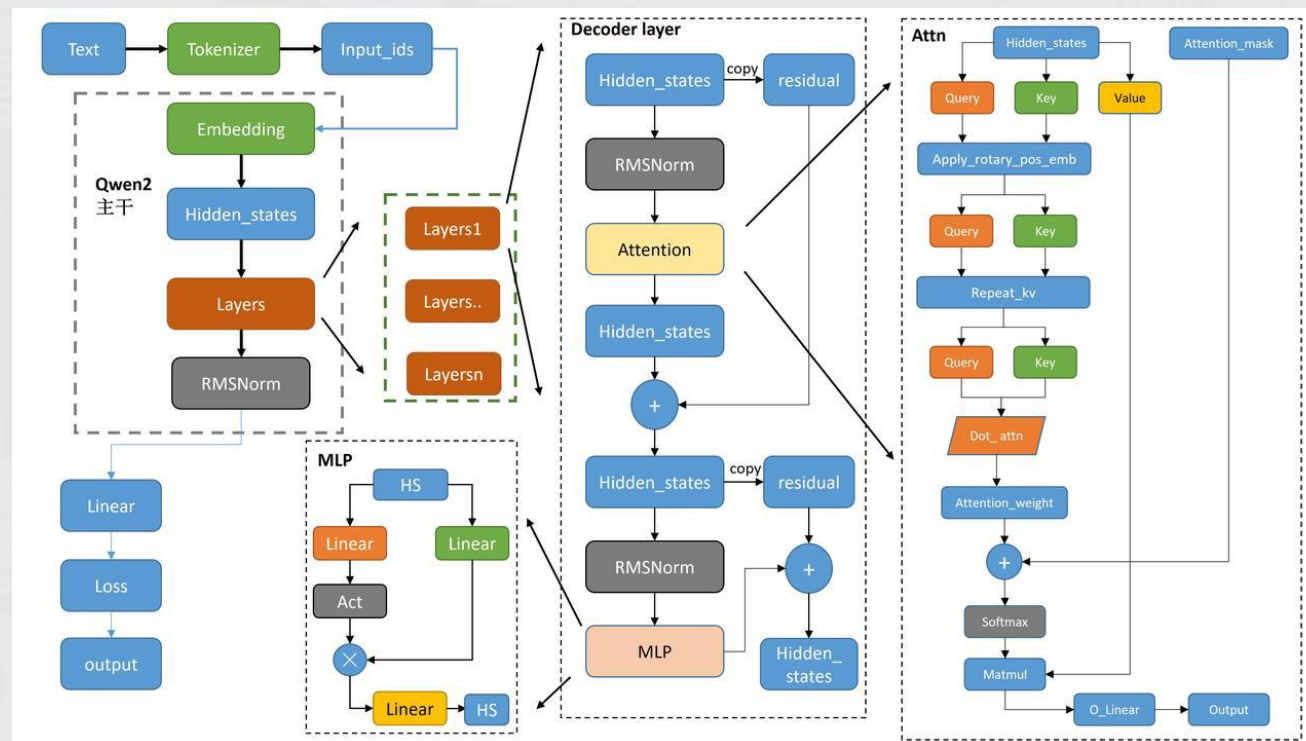


Qwen模型

通义千问是由阿里云自主研发的大模型，用于理解和分析用户输入的自然语言，以及图片、音频、视频等多模态数据。

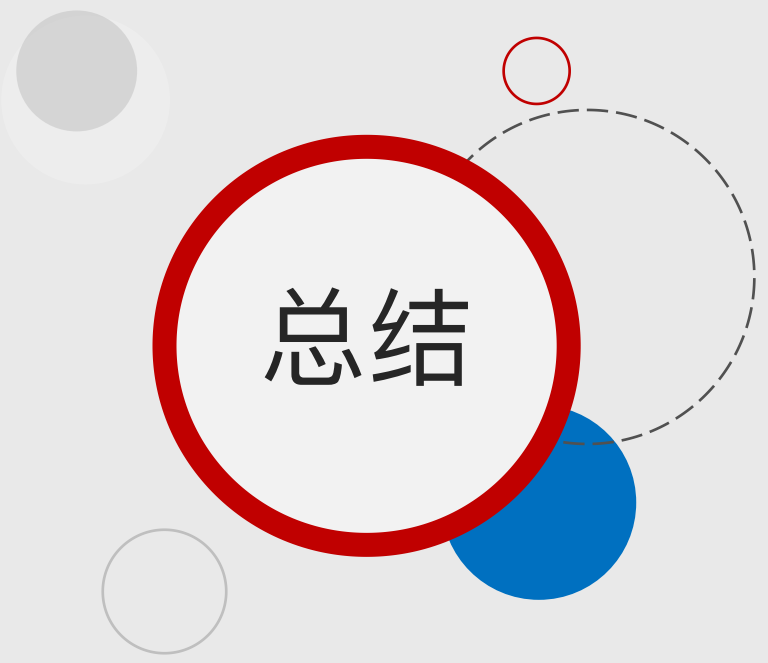
Qwen模型也是 Decoder-only架构，但结合前人的工作做了一些改进，比如：

- 位置编码：使用 RoPE (Rotary Positional Embedding)，增强长文本建模能力。
- 归一化层：采用 RMSNorm，替代传统 LayerNorm，提升训练稳定性。
- 激活函数：使用 SwiGLU，相比 GeLU 能更好地提取特征。



DeepSeek 模型是由杭州深度求索人工智能基础技术研究有限公司开发的一系列人工智能模型。该系列模型以 Transformer 架构为基础，核心采用创新的 DeepSeekMoE（混合专家）与 MLA（多头潜在注意力）技术路线，覆盖通用语言、代码、视觉、数学等多个领域。

模型名称	核心定位	关键能力
DeepSeek V3	通用大语言模型	具备优秀的多语言对话与知识理解能力，可处理复杂自然语言任务，适配多场景文本需求
DeepSeek R1	专业代码模型	支持 Python、Java 等多种编程语言，能实现代码生成、补全、优化及自动化测试用例生成，提升开发效率
DeepSeek-VL2	多模态视觉语言模型	采用 MoE 架构，可完成图像识别、视觉问答、文档 / 表格解析等任务，相比前代模型在多模态理解能力上大幅提升
DeepSeek-Prover-V2	数学与科学证明模型	拥有强大的逻辑推理和形式化证明能力，能协助解决复杂数学问题、完成定理推导，适配科研与教育场景
Janus-Pro	企业级定制模型	具备多语言能力和行业知识，支持定制化训练与部署，可为不同垂直领域的商业场景提供解决方案



总结

主流的开源大模型主要有：

- LLaMA, Meta AI 2023发布, 偏向英语系
- ChatGLM, 清华发布, 类ChatGPT
- Qwen, 阿里发布, 时下流行
- DeepSeek, 深度求索发布

AI 大模型

RAG与智能体开发

当前章节：《提示词工程》

大模型prompt工程指南

- 了解什么是提示词工程
- 掌握提示工程的设计技巧

提示词工程概念

提示工程（Prompt Engineering），也称为 In-Context Prompting，是指在**不更新模型权重**的情况下如何与大模型交互以引导其行为以获得所需结果的方法。



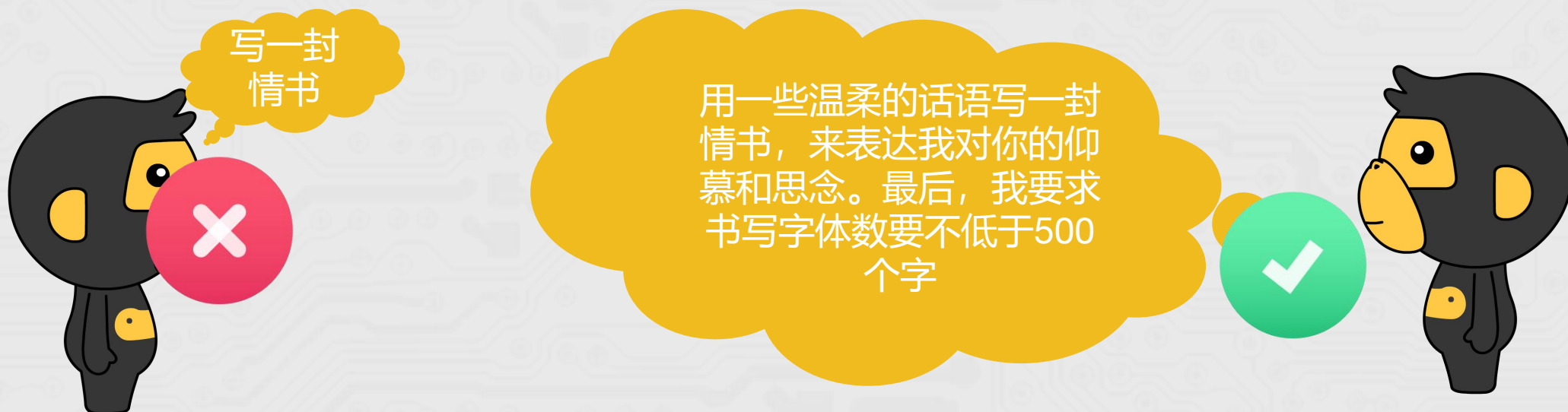
在人工智能领域，Prompt指的是用户给大型语言模型发出的指令。

- 例如，“「讲个笑话」”、“「用Python编个贪吃蛇游戏」”、“「写封情书」”等。
- 虽然看似简单，但实际上，Prompt的设计对于模型的结果影响很大。
- 因此如何设计prompt，进而与模型更好的交互，是研究人员必备的必不可少的技能（提示工程）。

提示词技巧

任何Prompt技巧，都不如清晰的表达你的需求。这就类似人与人沟通，如果话说不明白，不可能让别人理解你的思想。因此，写出清晰的指令，是核心。

技巧1：详细的描述



提示词技巧

任何Prompt技巧，都不如清晰的表达你的需求。这就类似人与人沟通，如果话说不明白，不可能让别人理解你的思想。因此，写出清晰的指令，是核心。

技巧2：让模型充当某个角色



我需要你充当一个AI算法面试官的角色，要求你自主的对我进行AI面试过程中常考的面试题，你可以一次说一个问题，然后我回答完，你再出第二道题



提示词技巧

技巧3：使用分隔符标明输入的不同部分

中括号、XML标签、三引号等分隔符可以帮助划分要区别对待的文本，也可以帮助模型更好的理解文本内容。常用`"""`把内容框起来



用20个字符总结由三引号分割的文本。`"""`在此插入文本`"""`



提示词技巧

技巧4：对任务指定步骤

对于可以拆分的任务可以尽量拆开，最好能为其指定一系列步骤，明确步骤可以让模型更容易实现它们。



利用下面分步情况来响应用户的输入。

步骤1: ""用户输入文本"", 用一句话总结这段文本, 并加上前缀"Summary".

步骤2: 将步骤1中的摘要翻译成英语, 并添加前缀"翻译: "



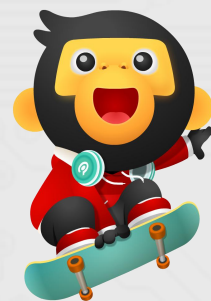
提示词技巧

技巧5：提供例子

本质类似于few-shot leaning。先扔给大模型举例，然后让模型按照例子来输出



按照这句评论文本的格式：'""用户输入
文本""，帮我创造新的样本



提示词技巧

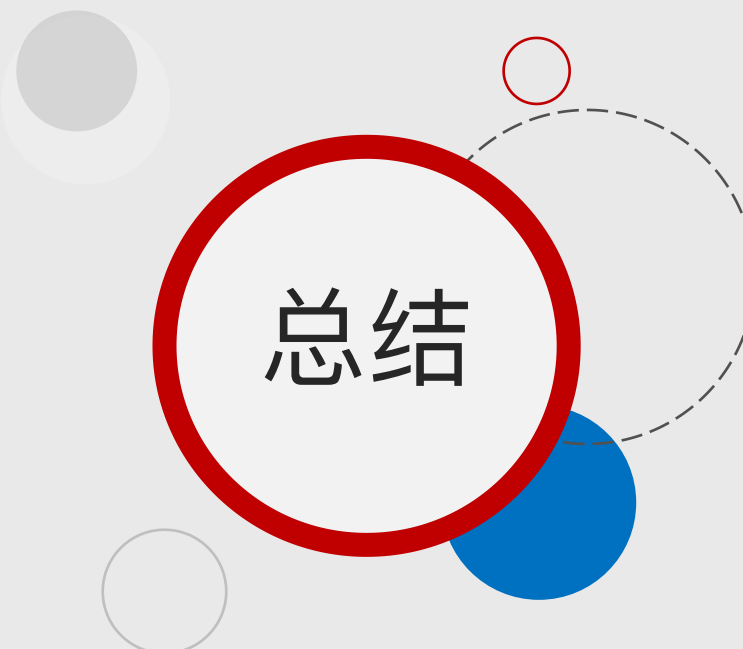
基于文本文档，辅助大模型问答，**降低模型“幻觉”**（一本正经的胡说八道）问题。

即使用参考文本作答 经典的知识库用法，让大模型使用我们提供的信息来组成答案。



根据下文中三重引号引起来的文章来回答问题。
如果在文章中找不到答案，请写“我找不到答案”，
不要自己造答案。
"""<在此插入文档>""""""<在此插入文档>""" 问
题：<在此插入问题>





总结

提示词工程就是更好的向模型提问的技巧。

大模型本身是一种很简单的结构，即：**用户输入，模型输出。**

- 用更详细、更清晰、有逻辑、有参考的提问，获得期望中的回答效果
- 不管是RAG还是Agent智能体亦或是其它围绕模型的各类复杂的开发工作，本质上都可以简单总结为在提示词上下功夫。

提示词优化是所有大模型应用开发的基础必修课，一个好的提示词，甚至能让基础模型的输出效果媲美经过简单微调的模型。

AI 大模型

RAG与智能体开发

当前章节：《提示词工程》

提示词实战案例介绍

- 了解案例的背景和整体任务
- 掌握Zero-shot、Few-shot的思想

实战案例背景

当前金融领域信息化发展的时代,金融数据大量激增,许多投资者和研究者试图通过对这些数据进行深度分析而获得一些有效的决策和帮助,尽可能减少决策失误带来的损失。

所以,针对金融数据的分析方法研究是目前十分有益且热门的话题。

当前案例主要有三大业务场景实现:

- 基于大模型完成: 金融文本分类
- 基于大模型完成: 金融文本信息抽取
- 基于大模型完成: 金融文本匹配

大模型选择: Qwen在线大模型 (阿里云通义千问 qwen3-max)

采用方法: 基于Few-Shot + Zero-Shot的思想, 设计prompt (提示词), 进而应用大模型完成相应的任务



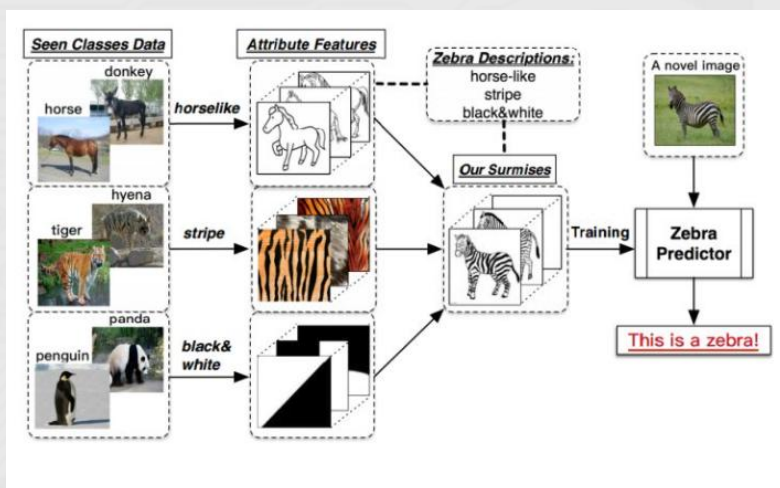
Zero-shot思想

Zero-shot学习 (Zero-shot Learning) 是指在训练阶段不存在与测试阶段完全相同的类别，但是模型可以使用训练过的知识来推广到测试集中的新类别上。

这种能力被称为“**零样本**”学习，因为模型在训练时从未见过测试集中的新类别，在模型训练和提示词优化中均有体现。

在模型训练中：

- 已知马（四脚兽）、虎（有条纹）、熊猫（黑白色）的特征，但未训练过斑马的数据（不认识）
- 告知模型：斑马是四脚兽、有黑白色的条纹
- 模型可以在已知数据中进行推理，从而识别斑马。



在提示词优化中：

- Zero-shot思想用于**基于已训练的能力**，不提供任何示例，仅通过语言去描述任务的要求、目标和约束，让模型**直接生成结果**。

简单来说就是“用语言定义任务，解放（信任）模型的预训练知识”

比如：
请判断””包围的用户评论中的情感倾向，输出 正面 或 负面。
”这款代餐鸡胸肉饱腹感很强，吃起来也不柴，很推荐！”

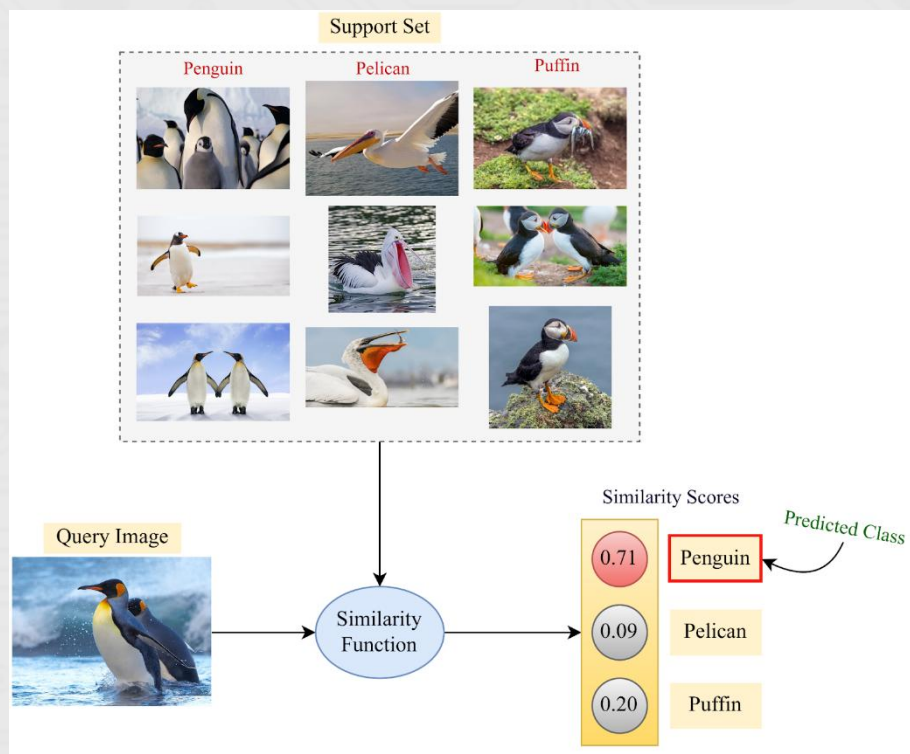


Few-shot思想

Few-shot学习 (Few-shot Learning) 是指少样本学习，当模型在学习了一定类别的大量数据后，对于新的类别，只需要少量的样本就能快速学习，对应的有one-shot learning，单样本学习，也算样本少到为一的情况下的一种few-shot learning。

在模型训练中（相似度判断方法）：

- 基于少量企鹅样本并结合相识度判断，推论未知图片内含“企鹅”



在提示词优化中：

- Few-shot主要用于基于少量示例，让模型参考示例回答。

简单来说就是“用示例定义任务，在模型的预训练知识的基础上，提升模型回答的对齐精度（比如参考示例的格式）”

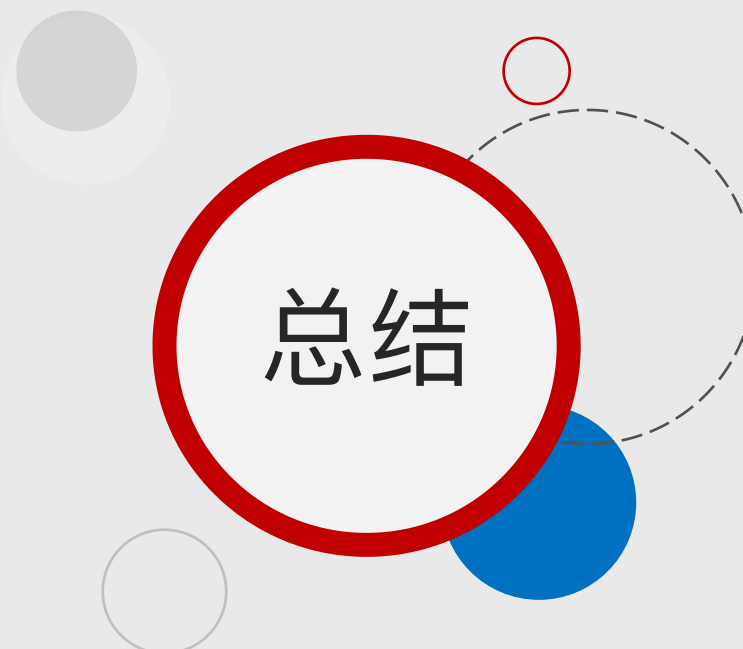
比如：

请抽取产品名称和核心卖点2个字段，格式为Json，我提供2个示例。

- 示例1: MacBookPro高效节能, 性能强大, 适合牛马工作使用
输出: {"产品名称": "MacBookPro", "产品卖点": "高效节能, 性能强大"}
 - 示例2: 联想笔记本拥有RTX4060独立显卡, 畅玩游戏, 丝滑流畅
输出: {"产品名称": "联想笔记本", "产品卖点": "畅玩游戏, 丝滑流畅"}
- 请处理: 华为MatepadPro, 高清大屏, 长效续航, 你的好帮手。

{"产品名称": "华为MatepadPro", "产品卖点": "高清大屏, 长效续航"}





总结

提示词工程实战的业务需求是在金融业务内完成：

- 文本分类
- 信息抽取
- 文本匹配

共3个任务目标，通过zero-shot、few-shot思想设计提示词完成。

在模型训练层面：

- Zero-shot：零样本，基于模型训练阶段学习的属性/语义关联，去迁移到未知的新类别
- Few-shot：少样本，基于少量样本，快速泛化识别新样本

在提示词优化层面：

- Zero-shot：无提示，语言描述任务，依赖模型预训练知识回答
- Few-shot：给与模型少量示例，引导模型对齐示例输出结果

AI 大模型

RAG与智能体开发

当前章节：《提示词工程》

提示词实战案例

LLM实现金融文本分类

- 掌握Fewshot方式下prompt的设计方式
- 掌握利用LLM实现文本分类的代码

LLM文本分类任务介绍

下面几段文本来自某平台发布的金融领域文本：

- 1."今日，央行发布公告宣布降低利率，以刺激经济增长。这一降息举措将影响贷款利率，并在未来几个季度内对金融市场产生影响。"
- 2."ABC公司今日发布公告称，已成功完成对XYZ公司股权的收购交易。本次交易是ABC公司在扩大业务范围、加强市场竞争力方面的重要举措。据悉，此次收购将进一步巩固ABC公司在行业中的地位，并为未来业务发展提供更广阔的发展空间。详情请见公司官方网站公告栏"
- 3."公司资产负债表显示，公司偿债能力强劲，现金流充足，为未来投资和扩张提供了坚实的财务基础。"
- 4."最新的分析报告指出，可再生能源行业预计将在未来几年经历持续增长，投资者应该关注这一领域的投资机会"

我们的目的是期望模型能够帮助我们识别出这4段话中，每一句话描述的是一个什么类型的报告。

即期望的输出结果为：['新闻报道', '公司公告', '财务公告', '分析师报告']



Prompt设计

对于大模型来讲，prompt 的设计非常重要，一个 明确 的 prompt 能够帮助我们更好从大模型中获得我们想要的结果。

在该任务的 prompt 设计中，我们主要考虑 2 点：

- 需要向模型解释什么叫作「文本分类任务」
- 需要让模型按照我们指定的格式输出

为了让模型知道什么叫做「文本分类」，我们借用 FewShot 的方式，给模型展示一些正确的例子：

User: "今日，股市经历了一轮震荡，受到宏观经济数据和全球贸易紧张局势的影响。投资者密切关注美联储可能的政策调整，以适应市场的不确定性。" 是['新闻报道', '公司公告', '财务公告', '分析师报告']里的什么类别？

Bot: 新闻报道

User: "本公司年度财务报告显示，去年公司实现了稳步增长的盈利，同时资产负债表呈现强劲的状况。经济环境的稳定和管理层的有效战略执行为公司的健康发展奠定了基础。"是['新闻报道', '公司公告', '财务公告', '分析师报告']里的什么类别？

Bot: 财务报告

其中，User 代表我们输入给模型句子，Bot 代表模型的回复内容。

注意：上述例子中 Bot 的部分也是由人工输入的，其目的是希望看到在看到类似 User 中的句子时，模型应当做出类似 Bot 的回答。

分类任务代码实现

实现llm_classification.py代码脚本文件，主要包含：init_prompts()和qwen_model() 2个核心函数

导入工具包：

```
from openai import OpenAI

# 1. 获取client对象, OpenAI类对象
client = OpenAI(
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
)

examples_data = {
    # 示例数据
    '新闻报道': '今日，股市经历了一轮震荡，受到宏观经济数据和全球贸易紧张局势的影响。投资者密切关注美联储可能的政策调整，以适应市场的不确定性。',
    '财务报告': '本公司年度财务报告显示，去年公司实现了稳步增长的盈利，同时资产负债表呈现强劲的状况。经济环境的稳定和管理层的有效战略执行为公司的健康发展奠定了基础。',
    '公司公告': '本公司高兴地宣布成功完成最新一轮并购交易，收购了一家在人工智能领域领先的公司。这一战略举措将有助于扩大我们的业务领域，提高市场竞争力',
    '分析师报告': '最新的行业分析报告指出，科技公司的创新将成为未来增长的主要推动力。云计算、人工智能和数字化转型被认为是引领行业发展的关键因素，投资者应关注这些趋势'
}

# 分类列表
examples_types = ['新闻报道', '财务报告', '公司公告', '分析师报告']

# 提问数据
questions = [
    "今日，央行发布公告宣布降低利率，以刺激经济增长。这一降息举措将影响贷款利率，并在未来几个季度内对金融市场产生影响。",
    "ABC公司今日发布公告称，已成功完成对XYZ公司股权的收购交易。本次交易是ABC公司在扩大业务范围、加强市场竞争力方面的重要举措。据悉，此次收购将进一步巩固ABC公司在行业中的地位，并为未来业务发展提供更广阔的发展空间。详情请见公司官方网站公告栏",
    "公司资产负债表显示，公司偿债能力强劲，现金流充足，为未来投资和扩张提供了坚实的财务基础。",
    "最新的分析报告指出，可再生能源行业预计将在未来几年经历持续增长，投资者应该关注这一领域的投资机会",
    "小明喜欢小新哟"
]
```



分类任务代码实现

构建init_prompts()函数，将准备好的示例数据，添加到list中，作为历史对话消息存在，基于FewShot模式提示模型

```
def init_prompt(examples):  
    prompt_list = [  
        {"role": "system", "content": f"你是一个金融专家，需要对输入的金融领域文  
本进行分析，将类型归类到examples_types，不在这四类的数据，输出'不清楚类型'，以下是几  
个示例分类："}  
    ]  
  
    for class_type, example_text in examples.items():  
        prompt_list.append({"role": "user", "content": example_text})  
        prompt_list.append({"role": "assistant", "content": class_type})  
  
    return prompt_list
```



分类任务代码实现

调用模型

```
if __name__ == '__main__':  
    messages = init_prompt(examples_data)  
  
    for q in questions:  
        response = client.chat.completions.create(  
            model="qwen3-max",  
            messages=messages + [{"role": "user", "content": f"请分析如下句子是什么类别: {q}" }],  
            stream=False  
        )  
  
        print(response.choices[0].message.content)
```



AI 大模型

RAG与智能体开发

当前章节：《提示词工程》

Json数据格式

Json数据格式

JSON (JavaScript Object Notation) 是一种轻量级的数据交换格式, 易于人阅读和编写, 同时也易于机器解析和生成。

Json是带有格式的字符串, 主要用于数据交换, 即程序和程序之间的信息互传, 使用Json会更加方便, 如下示例:

```
1 周杰轮11岁是个男孩
2 蔡依临12岁是个女孩
3 小明16岁是一个男孩子
```

Text文本

- 非结构化
- 抽取信息不方便

```
1 周杰轮,11,男
2 蔡依临,12,女
3 小明,16,男
```

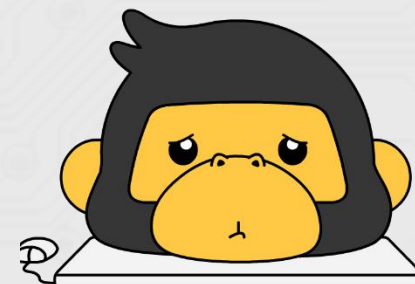
CSV (固定分隔符) 文本

- 结构化
- 抽取信息方便
- 数据不含Schema, 有一定风险

```
1 {
2     "name": "周杰轮",
3     "age": 11,
4     "gender": "男"
5 }
6 {
7     "name": "蔡依临",
8     "age": 12,
9     "gender": "女"
10 }
11 {
12     "name": "小明",
13     "age": 16,
14     "gender": "男"
15 }
```

Json格式文本

- 结构化
- 抽取信息方便
- 每条数据都包含Schema
- 空间占用大



Json数据格式

Json主要有2种结构：Json对象和Json数组

Json对象

```
{  
    "key": value,  
    "key": value  
}
```

- key必须是字符串
- value可以是：
 - 数字
 - 字符串
 - 列表
 - Json对象或Json数组

```
{  
    "age": 11,  
    "name": "周杰轮",  
    "hobby": ["唱", "跳", "rap"],  
    "other": {  
        "身高": 172,  
        "体重": 68  
    }  
}
```

Json数组

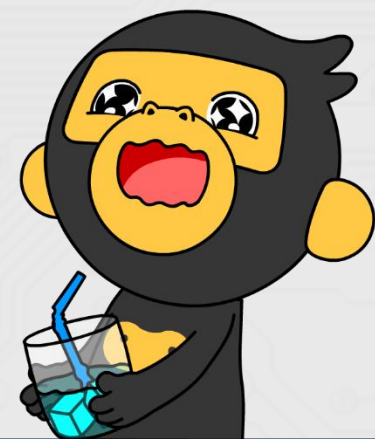
```
[{ }, { }, { }, ...]
```

- 即一堆Json对象的组合体

```
[  
    { "name": "周杰轮", "age": 11 },  
    { "name": "蔡依临", "age": 12 },  
    { "name": "小明", "age": 16 }  
]
```

- Json对象 => Python字典
- Json数组 => Python列表内含多个字典

Json在Python中，就是字典和列表套字典的字符串表现形式。



Python中使用Json

Python中使用Json主要完成：

- 将Python字典、列表转换为Json字符串
- 读取Json字符串，转换为Python字典或列表

主要使用Python内置的json库

- `json.dumps(字典或列表, ensure_ascii=False)`：将字典或列表转换为Json字符串
 - `ensure_ascii`参数确保中文能正常显示
 - 返回值：Json字符串
- `json.loads(json字符串)`：将Json字符串转换为Python字典或列表
 - 返回值：Python字典 或 Python列表



Json是一种带有格式的字符串，可以很好的记录数据和结构

- 便于不同程序之间数据交互

Json作为字符串，其内容和Python的数据结构可以有所对应。

- Json对象 → Python字典

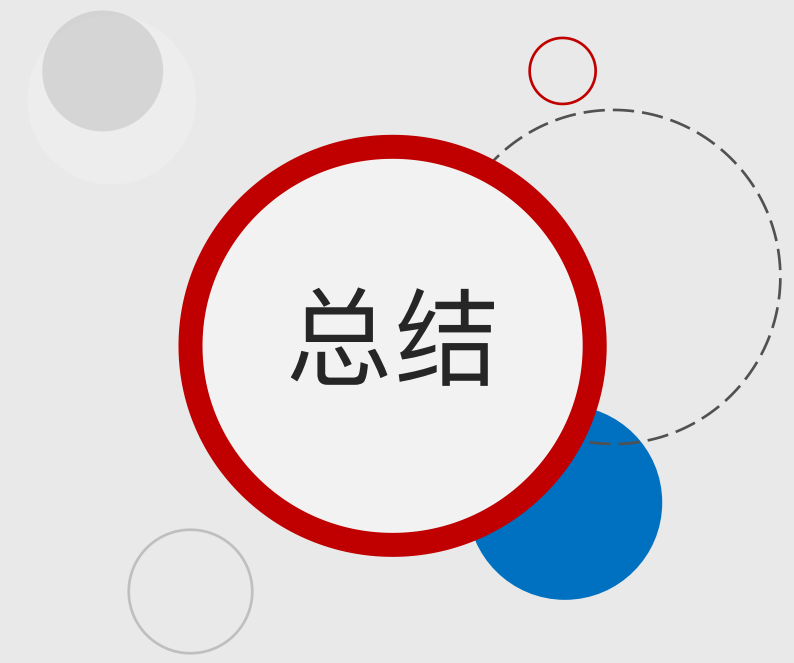
```
{  
    "age": 11,  
    "name": "周杰轮",  
    "hobby": ["唱", "跳", "rap"],  
    "other": {  
        "身高": 172,  
        "体重": 68  
    }  
}
```

- Json数组 → Python列表（内含多个字典）

```
[  
    {"name": "周杰轮", "age": 11},  
    {"name": "蔡依临", "age": 12},  
    {"name": "小明", "age": 16}  
]
```

import json

- json.dumps(字典或列表, ensure_ascii=False)
 - 将Python数据结构转为Json字符串
- json.loads(json字符串)
 - 将Json字符串转为Python数据结构



总结

AI 大模型

RAG与智能体开发

当前章节：《提示词工程》

提示词实战案例

LLM实现金融文本信息抽取

LLM信息抽取任务介绍

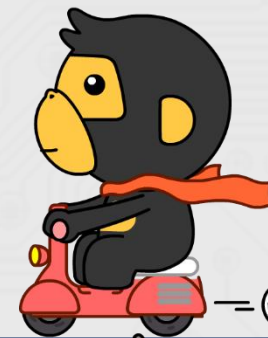
首先，我们定义信息抽取的Schema：

```
# 定义不同实体下的具备属性
schema = {
    '金融': ['日期', '股票名称', '开盘价', '收盘价', '成交量'],
}
```

下面几段文本来自某平台发布的股票信息：

- 1.'2023-02-15，寓意吉祥的节日，股票佰笃[BD]美股开盘价10美元，虽然经历了波动，但最终以13美元收盘，成交量微幅增加至460,000，投资者情绪较为平稳。'
- 2.'2023-04-05，市场迎来轻松氛围，股票盘古(0021)开盘价23元，尽管经历了波动，但最终以26美元收盘，成交量缩小至310,000，投资者保持观望态度。'

我们的目的是期望模型能够帮助我们识别出这2段话中的SPO三元组信息。



Prompt设计

在该任务的 prompt 设计中，我们主要考虑 2 点：

- 需要向模型解释什么叫作「信息抽取任务」
- 需要让模型按照我们指定的格式（json）输出

为了让模型知道什么叫做「信息抽取」，我们借用 FewShot 的方式，先给模型展示几个正确的例子：

User: '2023-01-10, 股市震荡。股票古哥-D[EOOE]美股今日开盘价100美元，一度飙升至105美元，随后回落至98美元，最终以102美元收盘，成交量达到520000。'。提取上述句子中“金融”('日期', '股票名称', '开盘价', '收盘价', '成交量')类型的实体，并按照JSON格式输出，上述句子中没有的信息用['原文中未提及']来表示，多个值之间用','分隔。

Bot: {'日期': ['2023-01-10'], '股票名称': ['古哥-D[EOOE]美股'], '开盘价': ['100美元'], '收盘价': ['102美元'], '成交量': ['520000']} ...

其中，User 代表我们输入给模型句子，Bot 代表模型的回复内容。

注意：上述例子中 Bot 的部分也是由人工输入的，其目的是希望看到在看到类似 User 中的句子时，模型应当做出类似 Bot 的回答。



信息抽取任务代码实现

本次信息抽取任务实现的主要过程：构造prompt -> 对句子做分类 -> 进行信息抽取

导入必备的工具包：

```
from dotenv import load_dotenv
import openai
import json
import re
import os

### 加载环境变量
load_dotenv()
# 定义不同实体下的具备属性
schema = {
    '金融': ['日期', '股票名称', '开盘价', '收盘价', '成交量'],
}
IE_PATTERN = "{}\n\n提取上述句子中{}的实体，并按照JSON格式输出，上述句子中不存在的信息用['原文中未提及']来表示，多个值之间用','分隔。"
# 提供一些例子供模型参考
ie_examples = {
    '金融': [
        {
            'content': '2023-01-10，股市震荡。股票古哥-D[E00E]美股今日开盘价100美元，一度飙升至105美元，随后回落至98美元，最终以102美元收盘，成交量达到520000。',
            'answers': {
                '日期': ['2023-01-10'],
                '股票名称': ['古哥-D[E00E]美股'],
                '开盘价': ['100美元'],
                '收盘价': ['102美元'],
                '成交量': ['520000'],
            }
        }
    ]
}
```

信息抽取任务代码实现

构建init_prompts()函数:

```
# 2 构造函数, 进行prompt设计
def init_prompt(ie_examples):
    """
    构建实体提取提示词
    :param ie_examples:
    :return:
    """
    history = [
        {'role': 'system', 'content': "现在你需要帮助我完成信息抽取任务, 当我给你一个句子时, 你需要帮我抽取出句子中实体信息, 并按照JSON的格式输出, 上述句子中没有的信息用['原文中未提及']来表示, 多个值之间用','分隔。"}
    ]
    for example_cls, examples in ie_examples.items():
        for example in examples:
            history.append({'role': 'user', 'content': IE_PATTERN.format(example['content'], schema[example_cls])})
            history.append({'role': 'assistant', 'content': json.dumps(example['answers'])})
    return history
```

信息抽取任务代码实现

构建clean_response()函数:

```
def clean_response(response: str):  
    """  
    后处理模型输出。  
    Args:  
        response (str): _description_  
    """  
    if '`json`' in response:  
        res = re.findall(r'`json(?:.*?)`', response)  
        if len(res) and res[0]:  
            response = res[0]  
            response.replace(' ', ',')  
    try:  
        return json.loads(response)  
    except:  
        return response
```

信息抽取任务代码实现

构建qwen_model()函数:

```
def qwen_model(prompt, client, history=[]):  
    """  
    构建qwen模型推理  
    :param prompt:  
    :param client:  
    :param history:  
    :return:  
    """  
    completion = client.chat.completions.create(  
        model=os.getenv("model"),  
        messages=history+ [{'role': 'user', 'content': prompt}]  
    )  
    resp = completion.choices[0].message.content  
    return resp
```

信息抽取任务代码实现

代码调用和结果展示:

```
if __name__ == '__main__':  
    history = init_prompt(ie_examples)  
    for sentence in sentences:  
        resp=qwen_model(IE_PATTERN.format(sentence,  
schema['金融']), client, history)  
        print(resp)  
        print(clean_response(resp))
```

```
D:\miniconda3\python.exe E:\work\czbk\ev\code\day04\finance_ie2.py  
```json  
{
 "日期": ["2023-02-15"],
 "股票名称": ["佰笃[BD]美股"],
 "开盘价": ["10美元"],
 "收盘价": ["13美元"],
 "成交量": ["460,000"]
}
...
```json  
{  
  "日期": ["2023-04-05"],  
  "股票名称": ["盘古(0021)"],  
  "开盘价": ["23元"],  
  "收盘价": ["26美元"],  
  "成交量": ["310,000"]  
}  
...
```



练习

彩票信息抽取作业

有如下5条文本，需要抽取信息：

1. 2025年第100期，开好红球22 21 06 01 03 11 篮球 07，一等奖中奖为2注。
2. 2025101期，有3注1等奖，10注2等奖，开号篮球11，中奖红球3、5、7、11、12、16。

期待经过通过模型抽取信息得到如下结果：

1. {"期数": "2025100", "中奖号码": [1, 3, 6, 11, 21, 22, 7], "一等奖": "2注"}
2. {"期数": "2025101", "中奖号码": [3, 5, 7, 11, 12, 16, 11], "一等奖": "3注"}

请设计Prompt，基于FewShot，对模型做示例的提示，得到需要的结果。

AI 大模型

RAG与智能体开发

当前章节：《提示词工程》

提示词实战案例

LLM实现金融文本匹配

LLM信息文本匹配任务介绍

首先，我们构造几个短文本对：

- 1.('股票市场今日大涨，投资者乐观。','持续上涨的市场让投资者感到满意。'),
- 2.('油价大幅下跌，能源公司面临挑战。','未来智能城市的建设趋势愈发明显。'),
- 3.('利率上升，影响房地产市场。','高利率对房地产有一定冲击。'),

我们期望模型能够帮我们识别出成对的句子中，2句话是否是有关联的。

我们期望模型输出的结果为：

['是', '不是', '是']



Prompt设计

在该任务的 prompt 设计中，我们主要考虑 2 点：

- 需要向模型解释什么叫作「文本匹配任务」
- 需要让模型按照我们指定的格式输出

为了让模型知道什么叫做「文本匹配任务」，我们借用 FewShot 的方式，先给模型展示几个正确的例子：

User:

句子一: 公司ABC发布了季度财报, 显示盈利增长。 \n

句子二: 财报披露, 公司ABC利润上升

Bot: 是

User:

句子一: 黄金价格下跌, 投资者抛售。 \n

句子二: 外汇市场交易额创下新高

Bot: 不是 ...

其中，User 代表我们输入给模型的句子，Bot 代表模型的回复内容。

注意：上述例子中 Bot 的部分也是由人工输入的，其目的是希望看到在看到类似 User 中的句子时，模型应当做出类似 Bot 的回答。



文本匹配任务代码实现

导入必备的工具包:

```
"""
利用 Qwen LLM 进行文本匹配
"""

# 1 导入必备的工具包
from dotenv import load_dotenv
from openai import OpenAI
import json
import re
import os

# 提供相似, 不相似的语义匹配例子
examples = {
    '是': [
        ('公司ABC发布了季度财报, 显示盈利增长。', '财报披露, 公司ABC利润上升。'),
    ],
    '不是': [
        ('黄金价格下跌, 投资者抛售。', '外汇市场交易额创下新高。'),
        ('央行降息, 刺激经济增长。', '新能源技术的创新。')
    ]
}
```

文本匹配任务代码实现

构建init_prompts()函数:

```
# 2 构造函数, 进行prompt设计 (描述清楚任务及输出格式)
def init_prompt(examples):
    """
    定义匹配的history
    :param examples:
    :return:
    """
    history = [
        {'role': 'system', 'content': '判断2个句子语义是否相似, 相似句子返回是, 不相似句子返回不是, 不要其他回答'},
    ]
    for label, example in examples.items():
        for sentence_pair in example:
            history.append({'role': 'user', 'content': "句子一: {}\n句子二: {}\n 是否是语义相同的句子?".format(sentence_pair[0],
sentence_pair[1])})
            history.append({'role': 'assistant', 'content': label})
    return history
```

文本匹配任务代码实现

构建qwen_model()函数:

```
def qwen_model(prompt, client, history=[]):  
    """  
    qwen模型推理  
    """  
    completion = client.chat.completions.create(  
        model=os.getenv("model"),  
        messages=history+[{ 'role': 'user', 'content': prompt}]  
    )  
    return completion.choices[0].message.content
```

文本匹配任务代码实现

代码调用和结果打印展示:

```
if __name__ == '__main__':  
    history = init_prompt(examples)  
    for pairs in sentence_pairs:  
        print(pairs)  
        prompt = "句子一: {}\n句子二: {}\n 是否是语义相同的句子?"  
        ".format(pairs[0], pairs[1])  
        response = qwen_model(prompt, client, history)  
        print(response)
```

```
D:\miniconda3\python.exe E:\work\czbk\ev\code\day04\finance_pair2.py  
( '股票市场今日大涨，投资者乐观。', '持续上涨的市场让投资者感到满意。' )  
是  
( '油价大幅下跌，能源公司面临挑战。', '未来智能城市的建设趋势愈发明显。' )  
不是  
( '利率上升，影响房地产市场。', '高利率对房地产有一定冲击。' )  
是
```

AI 大模型

RAG与智能体开发

当前章节：《提示词工程》

课后作业

提示词优化作业

代码调用和结果打印展示:

```
if __name__ == '__main__':  
    history = init_prompt(examples)  
    for pairs in sentence_pairs:  
        print(pairs)  
        prompt = "句子一: {}\n句子二: {}\n 是否是语义相同的句子?"  
        ".format(pairs[0], pairs[1])  
        response = qwen_model(prompt, client, history)  
        print(response)
```

```
D:\miniconda3\python.exe E:\work\czbk\ev\code\day04\finance_pair2.py  
( '股票市场今日大涨，投资者乐观。', '持续上涨的市场让投资者感到满意。' )  
是  
( '油价大幅下跌，能源公司面临挑战。', '未来智能城市的建设趋势愈发明显。' )  
不是  
( '利率上升，影响房地产市场。', '高利率对房地产有一定冲击。' )  
是
```